BTS SIO - Option SLAM | 1ère Année | 2025 - 2026

RAPPORT DE STAGE

du 18 mai au 24 juin 2026

Hugo GANGNEUX

Lycée Paul-Louis Courier - Tours

Tuteur pédagogique : Mme Christèle Ferreira-Mercier

Stage effectué chez

SECTRONIC
GROUPE SYNOV

18 rue des Granges Galand - 37550 Saint-Avertin

Maître de stage : Olivier VANHOUCKE

Directeur Technique & Référent Système d'Information

Conception et fabrication de cartes et équipements électroniques depuis 1975

Remerciements

Je tiens à adresser mes sincères remerciements à toutes les personnes qui ont contribué au bon déroulement de ce stage.

Je remercie en premier lieu mon maître de stage M. Olivier Vanhoucke, Directeur Technique et Référent Système d'Information chez SECTRONIC, qui a accepté de m'accueillir au sein du Bureau d'Études en tant que maître de stage. Sa disponibilité, ses conseils techniques et sa confiance m'ont permis de prendre rapidement en main des missions concrètes et de progresser rapidement tout au long du stage.

Je remercie également l'ensemble de l'équipe du Bureau d'Études de SECTRONIC pour leur accueil chaleureux, leur patience et leur bienveillance. Chacun a contribué, à sa manière, à rendre cette expérience enrichissante : que ce soit en répondant à mes questions, en m'expliquant le fonctionnement des projets ou simplement en m'intégrant à la vie de l'équipe au quotidien.

Enfin, je remercie Mme Christèle Ferreira-Mercier, mon tuteur pédagogique au lycée Paul-Louis Courier, pour son suivi régulier et ses conseils tout au long de la période de stage.

Sommaire

Table des matières

Remerciements....2
Sommaire....3
1. Présentation du contexte....4
1.1 Présentation de l'entreprise....4
1.2 Présentation du service dans lequel j'ai été accueilli....6
1.3 Présentation de l'environnement matériel et logiciel....6
1.4 Présentation des missions....8
1.5 Gestion de projet....8
2. Réalisation des missions....10
2.1 Mission 1 : Inventaire du patrimoine informatique....10
2.2 Mission 2 : Développement de l'application Visiteur....12
2.3 Mission 3 : Développement de l'application de gestion des congés....20
2.4 Mission 4 : Tests de l'application M-EXTEND....31
3. Bilan du stage....36
3.1 Les objectifs....36
3.2 Compétences techniques acquises....36
3.3 Compétences professionnelles développées....37
3.4 Difficultés rencontrées....37
3.5 Apports du stage....38
3.6 Conclusion générale....39
ANNEXES....40

1. Présentation du contexte

1.1 Présentation de l'entreprise

1.1.1 Historique et activité

J'ai effectué mon stage au sein de la société SECTRONIC, dont le Bureau d'Études est situé au 18 rue des Granges Galand à Saint-Avertin (37550) et le site de production au ZI Les Saulniers à Sainte-Maure-de-Touraine (37800).

Créée en 1975 sous le nom « Société Electronique du Centre », SECTRONIC est une entreprise spécialisée dans la conception, l'industrialisation et la fabrication de cartes et d'équipements électroniques. Depuis plus de 50 ans, elle propose une offre globale de sous-traitance électronique, de l'idée initiale jusqu'à l'intégration du produit fini, en accompagnant ses clients tout au long du cycle de vie de leurs projets.

En 2019, SECTRONIC a rejoint le groupe SYNOV, groupe industriel français spécialisé dans les solutions électroniques sur mesure. Elle est aujourd'hui dirigée par M. Cédric Desrier, Directeur Général et Directeur Commercial.

1.1.2 Chiffres clés

SECTRONIC emploie environ 80 collaborateurs répartis entre ses deux sites. Elle réalise plusieurs millions d'euros de chiffre d'affaires et dispose d'un parc machine régulièrement renouvelé, notamment par une extension bâtimentaire réalisée en 2024 avec l'évolution du parc machines.

1.1.3 Secteurs d'activité

SECTRONIC intervient dans de nombreux secteurs industriels exigeants, ce qui impose des standards de qualité élevés à l'ensemble de ses processus de fabrication :

- Transport et ferroviaire (carte de pilotage TGV, interfaces homme-machine).
- Médical (cartes d'alimentation et de contrôle).
- Industrie et instrumentation (plateformes IHM tactiles, convertisseurs).
- Défense et sécurité (contrôle d'accès nucléaire EDF).
- Énergie (systèmes BMS, convertisseurs statiques d'alimentation).
- Domotique, télécommunications et nucléaire.

1.1.4 Certifications

Pour répondre aux exigences de ses marchés, SECTRONIC est certifiée ISO 9001 et membre de l'IPC (Association des Industries Electroniques). Elle dispose

également des certifications UL et d'une qualification EDF UTO, indispensables pour intervenir dans le secteur nucléaire.

1.1.5 Historique de l'entreprise

Depuis sa création, SECTRONIC a connu plusieurs étapes clés dans son développement :

Année	Événement
1975	Création de SECTRONIC (« Société Electronique du Centre »).
1980	Installation dans les locaux à Sainte-Maure-de-Touraine (1 500 m²).
1985	Création du Bureau d'Études.
1994	1ère certification AFAQ et AQF 2 par la SNCF.
2001	Intégration au groupe ELECTRONAVALE.
2004	Agrandissement des locaux (+1 000 m²).
2006	Rachat par le groupe MINERVA.
2009	Mise en service d'une inspection Rayon X.
2014	Nouvelle direction.
2018	Intégration d'une deuxième ligne CMS.
2019	Création du Bureau d'Études à Saint-Avertin. Intégration au groupe SYNOV.
2024	Extension bâtimentaire (+1 000 m²) et évolution du parc machines.

1.1.6 Organisation de l'entreprise

SECTRONIC est organisée autour de plusieurs services, placés sous l'autorité du Directeur Général M. Cédric Desrier et du Directeur de Site M. Philippe Batty. On peut distinguer les services suivants :

- Direction Générale et Direction Commerciale.
- Bureau d'Études : conception et développement des applications et systèmes électroniques.
- Service Méthodes : industrialisation des produits.
- Responsable Fabrication et Ordonnancement : pilotage de la production.
- Équipe Process « cartes » (PCBA) : assemblage et intégration des cartes électroniques.
- Technicien Maintenance.
- Référent Sécurité.
- Service Qualité et Sûreté Nucléaire.
- Service Achats et Approvisionnements.
- Administration des Ventes.

- Service Commercial.
- Administration et Finances.

Un organigramme hiérarchique (Annexe A.1) et fonctionnel, émis par M. Cédric Desrier le 19 mai 2026, est joint en annexe de ce rapport. Il détaille les noms des responsables de chaque service ainsi que les liens hiérarchiques entre eux.

1.2 Présentation du service dans lequel j'ai été accueilli

J'ai réalisé mon stage au sein du **Bureau d'Études de SECTRONIC**, situé à Saint-Avertin. Ce service est rattaché à la Direction Technique et est dirigé par **M. Olivier Vanhoucke**, Directeur Technique et Référent Système d'Information, qui a également été mon maître de stage.

Le Bureau d'Études regroupe une équipe d'ingénieurs, de développeurs et de techniciens intervenant sur l'ensemble du cycle de vie des produits électroniques. Ses compétences couvrent notamment le développement électronique analogique et numérique sous Altium Designer, le développement logiciel embarqué (Linux embarqué, FreeRTOS, µP/µC, CPLD/FPGA), les études mécaniques sous SolidWorks et SolidEdge, ainsi que le prototypage électronique et mécanique avec impression 3D. Le Bureau d'Études mène également des activités de recherche et développement autour des objets connectés, des technologies radio (LoRa, 4G, Bluetooth, RFID) et des systèmes embarqués à très basse consommation. Il assure enfin la qualification et les essais des produits selon les normes en vigueur, notamment les exigences ferroviaires EN 50155 et nucléaires RCC-E.

En parallèle de ces activités, le Bureau d'Études participe également au développement de certains outils internes utilisés par l'entreprise. La gestion du système d'information et du parc informatique est principalement assurée par M. Olivier Vanhoucke dans son rôle de Référent Système d'Information, avec l'appui de prestataires externes lorsque cela est nécessaire.

Durant mon stage, j'ai participé à plusieurs missions liées au système d'information de l'entreprise. J'ai d'abord réalisé un inventaire du patrimoine informatique avant de contribuer au développement de deux applications web destinées à être intégrées à l'intranet de l’entreprise.

1.3 Présentation de l'environnement matériel et logiciel

1.3.1 Environnement matériel

L'entreprise dispose d'une infrastructure informatique répartie sur ses deux sites. Elle s'appuie sur un serveur HPE ProLiant sous Windows Server 2016 pour l'hébergement des applications et des données, complété par un NAS QNAP pour le stockage réseau et une solution de sauvegarde Beemo pour la protection des données critiques. La sécurité réseau est assurée par un pare-feu Stormshield. La connectivité entre les équipements est garantie par plusieurs switchs TP-Link et Netgear répartis sur les deux sites. Les collaborateurs disposent de postes de travail

fixes et portables, ainsi que de mini PC connectés à des écrans de télévision pour l'affichage dynamique d'informations dans les locaux via le module SectroTV.

1.3.2 Environnement logiciel

L'environnement logiciel du Bureau d'Études combine des outils de communication et de collaboration, des outils de développement et des outils de gestion de projet :

- Mattermost : messagerie collaborative interne, utilisée pour la communication quotidienne au sein de l'équipe.
- Redmine : outil de gestion de tâches et de suivi de projets, permettant d'organiser les missions et de suivre leur avancement.
- GitLab : plateforme de gestion de versions, utilisée pour héberger les dépôts de code source et organiser le travail en branches.
- Visual Studio Code (VSCode) : environnement de développement intégré, utilisé pour l'écriture du code.
- draw.io : outil de création de schémas et de diagrammes, utilisé pour la cartographie du réseau.
- Microsoft Excel et LibreOffice Calc : utilisés pour la réalisation des tableaux d'inventaire du matériel informatique.
- Ainsi que d'autres outils et logiciels spécialisés utilisés en fonction des projets et des besoins des clients.

1.3.3 Environnement de développement

Dans le cadre de mes missions de développement, j'ai utilisé les technologies suivantes, directement intégrées au projet intranet de l'entreprise :

- Python 3.12 avec le framework Django : langage et framework utilisés pour le développement du portail intranet.
- HTML, CSS et Bootstrap 5 : technologies front-end utilisées pour l'interface utilisateur des applications web.
- JavaScript : utilisé pour les interactions dynamiques côté navigateur (formulaires, calendriers, canvas de signature, etc.).
- SQLite : base de données utilisée en environnement de développement local.
- SQL Server (Microsoft) : base de données de production, utilisée par le projet intranet en environnement déployé.
- FullCalendar : bibliothèque JavaScript intégrée pour l'affichage de calendriers interactifs.
- pymupdf : bibliothèque Python utilisée pour la génération de documents PDF.
- qrcode : bibliothèque Python utilisée pour la génération de QR codes.

1.4 Présentation des missions

Au cours de mon stage au sein de SECTRONIC, j'ai participé à plusieurs projets liés au développement et à l'évolution des outils internes de l'entreprise.

Mes missions principales ont été les suivantes :

- Réalisation d'un inventaire du patrimoine informatique et de l'infrastructure du site de Saint-Avertin et Saint-Maure.
- Développement de l'application « Visiteur », destinée à gérer l'accueil des visiteurs et les procédures d'évacuation au sein de l'entreprise.
- Développement de l'application « Congés », permettant de dématérialiser le processus de gestion des demandes d'absence via un circuit de validation hiérarchique à quatre niveaux (salarié, chef de service, secrétariat, direction générale).
- Participation aux phases de tests, de banc de test M-EXTEND.

Ces missions m'ont permis de mettre en pratique des compétences en développement web, gestion de bases de données, intégration d'API, tests logiciels et gestion de projet.

1.5 Gestion de projet

Pour mener à bien ces différentes missions dans les délais impartis, j'ai appliqué les méthodologies de gestion de projet en vigueur chez SECTRONIC, basées sur des cycles itératifs et un suivi rigoureux.

1.5.1 Outils de suivi et de collaboration

- Mattermost (Communication) : Cet outil a été le principal canal de communication avec mon maître de stage et l'équipe du Bureau d'Études, notamment pour poser des questions, faire des points d'avancement et recevoir les retours sur mon travail.

- Redmine (Gestion des tâches) : Cet outil était disponible dans l'environnement de travail de l'entreprise mais n'a pas été utilisé directement dans le cadre de mes missions.

- GitLab (Versionnage) : L'ensemble du code source a été géré via Git, hébergé sur le GitLab interne de l'entreprise. Pour chaque nouvelle mission de développement, j'ai créé une branche dédiée afin d'isoler mon travail du reste du projet sans perturber la branche principale. J'ai effectué des commits réguliers tout au long du développement, avec des messages explicites

décrivant les modifications apportées, ce qui permet de retrouver facilement l'historique des changements.

Lors de la finalisation de chaque application, j'ai procédé à l'intégration sur la branche principale (main) en suivant plusieurs étapes : fusion de ma branche en local, vérification du bon fonctionnement après fusion, récupération des commits effectués par les autres développeurs sur le dépôt distant, rebase pour intégrer ces évolutions, puis push final sur le dépôt GitLab de l'entreprise. Cette séquence correspond au workflow Git utilisé en entreprise sur un projet partagé, que j'ai pu appliquer concrètement pour la première fois.

1.5.2 Attribution et suivi des missions

Les missions ont été définies et attribuées de manière progressive au cours du stage. À chaque fin de mission, mon maître de stage m'en confiait une nouvelle en fonction des besoins de l'entreprise et de l'avancement du projet intranet. Cette organisation itérative m'a permis de m'adapter rapidement aux priorités de l'équipe.

2. Réalisation des missions

2.1 Mission 1 : Inventaire du patrimoine informatique

(Annexes B.1, B.2, B.3, B.4)

2.1.1 Contexte et besoin

À mon arrivée chez SECTRONIC, mon maître de stage, M. Olivier Vanhoucke, m'a confié une première mission consistant à réaliser un inventaire complet du patrimoine informatique des sites de Saint-Avertin et de Sainte-Maure-de-Touraine.

L'objectif était de disposer d'une vision précise et actualisée de l'infrastructure informatique de l'entreprise. Les informations existantes étaient incomplètes ou dispersées, ce qui rendait plus difficile le suivi du matériel, la gestion des interventions et l'évolution du système d'information.

Cette mission devait également permettre de produire une documentation technique utilisable par le Référent Système d'Information ainsi que par les prestataires intervenant sur l'infrastructure.

2.1.2 Objectifs

Les objectifs de cette mission étaient les suivants :

- Recenser l'ensemble des équipements informatiques présents sur les deux sites.
- Identifier les caractéristiques techniques de chaque équipement.
- Relever les adresses IP et les informations réseau.
- Vérifier la présence des étiquettes d'inventaire internes (HMB).
- Réaliser une cartographie de l'infrastructure réseau.
- Produire une documentation claire et facilement maintenable.

2.1.3 Outils utilisés

Pour réaliser cette mission, plusieurs outils ont été utilisés :

- Microsoft Excel et LibreOffice Calc pour la création du tableau d'inventaire.
- Draw.io pour la réalisation des schémas réseau.
- Interface d'administration des équipements réseau (switchs, routeurs, pare-feu).
- Outils Windows permettant l'identification des postes et des adresses IP.
- NETGEAR Discovery Tools pour la détection des équipements NETGEAR, et Omada Discovery Utility pour les équipements TP-LINK.

2.1.4 Réalisation

La mission a débuté par une phase de recensement physique des équipements présents sur les deux sites de l'entreprise.

Pour chaque matériel identifié, j'ai relevé :

- la désignation de l'équipement ;
- la marque et le modèle ;
- le numéro de série ;
- l'adresse IP lorsqu'elle était attribuée ;
- la localisation ;
- la présence de l'étiquette d'inventaire interne.

Les équipements recensés comprenaient notamment :

- les serveurs ;
- les équipements réseau (switchs, routeurs, pare-feu) ;
- les NAS ;
- les postes de travail ;
- les imprimantes ;
- les mini-PC utilisés pour l'affichage dynamique.

Une fois les informations collectées, j'ai structuré les données dans un tableau d'inventaire puis réalisé une cartographie réseau à l'aide de Draw.io afin de représenter les liens entre les différents équipements.

2.1.5 Difficultés rencontrées

L'une des principales difficultés rencontrées a été l'identification de certains équipements dont les informations n'étaient pas immédiatement accessibles.

Pour résoudre ce problème, j'ai consulté les interfaces d'administration des équipements ainsi que les paramètres réseau afin de compléter les informations manquantes.

Le cas le plus complexe a été l'identification du troisième switch (TP-LINK TL-SG2424P), qui n'était pas détecté par les outils de scan habituels car il était configuré sur un sous-réseau différent de celui de mon poste de travail. Pour le localiser, j'ai modifié temporairement l'adresse IP de mon poste afin de me placer dans le même sous-réseau que l'équipement, ce qui m'a permis de le pinguer et d'accéder à son interface d'administration. D'autres techniques ont également été utilisées, comme le ping broadcast suivi d'une consultation de la table ARP (commande `arp -a`) pour lister les adresses IP actives sur le réseau.

La réalisation du schéma réseau a également nécessité une phase d'analyse afin de comprendre l'organisation des connexions entre les différents matériels présents sur les deux sites.

2.1.6 Résultats obtenus

Cette mission a permis de produire :

- un inventaire complet du matériel informatique des deux sites ;
- une documentation centralisée des caractéristiques techniques des équipements ;
- une cartographie du réseau de l'entreprise ;
- une meilleure visibilité sur l'organisation de l'infrastructure informatique.

Les documents réalisés pourront être utilisés pour faciliter la maintenance, les futures évolutions de l'infrastructure et le suivi du patrimoine informatique de l'entreprise.

2.2 Mission 2 : Développement de l'application Visiteur

2.2.1 Contexte et besoin

Avant mon arrivée chez SECTRONIC, l'enregistrement des visiteurs était réalisé à l'aide d'un registre papier placé à l'accueil de chaque site. Cette méthode présentait plusieurs limites : les visiteurs oubliaient parfois de renseigner leur présence, certaines informations étaient difficilement lisibles et il n'était pas possible de connaître en temps réel les personnes présentes dans les locaux.

Cette situation devenait particulièrement problématique en cas d'évacuation d'urgence. Pour connaître les visiteurs présents dans un bâtiment, il fallait consulter manuellement le registre papier, ce qui pouvait entraîner des pertes de temps et des erreurs.

Afin de moderniser ce processus, SECTRONIC a souhaité développer une application web intégrée à son intranet permettant de remplacer le registre papier par une solution numérique accessible depuis une tablette Android installée à l'accueil de chaque site.

L'objectif principal du projet était de disposer en permanence d'une liste fiable des visiteurs présents sur chaque site tout en garantissant la prise de connaissance et l'acceptation des consignes de sécurité.

2.2.2 Analyse du besoin

Avant de commencer le développement, une phase d'analyse a été réalisée afin d'identifier précisément les besoins fonctionnels de l'application.

L'un des principaux enjeux concernait la gestion multi-sites. SECTRONIC possède plusieurs implantations disposant chacune de leur propre accueil. Les visiteurs enregistrés sur un site ne doivent jamais apparaître sur un autre. L'application a donc été conçue dès le départ comme une solution multi-sites, avec une tablette dédiée à chaque implantation.

Les fonctionnalités attendues étaient les suivantes :

- enregistrement autonome des visiteurs depuis une tablette tactile ;

- gestion centralisée de plusieurs sites ;
- consultation en temps réel des visiteurs présents ;
- affichage des consignes de sécurité adaptées au site concerné ;
- signature électronique obligatoire des consignes ;
- gestion du départ des visiteurs ;
- fermeture automatique des visites oubliées en fin de journée ;
- génération d'une liste d'évacuation en cas d'urgence ;
- envoi automatique d'un courrier électronique contenant la liste des personnes présentes.

L'application étant destinée à des visiteurs externes, parfois peu familiers avec les outils numériques, l'interface devait être simple, intuitive et optimisée pour une utilisation tactile sur tablette Android.

2.2.3 Technologies utilisées

Pour réaliser cette application, j'ai utilisé les technologies suivantes :

- Python 3.12 ;
- Django ;
- HTML5 ;
- CSS3 ;
- Bootstrap 5 ;
- JavaScript ;
- SQLite pour le développement ;
- SQL Server (préparation de la migration en environnement de production) ;
- GitLab pour la gestion de version ;
- Signature Pad pour la signature électronique ;
- bibliothèque Python qrcode pour la génération des QR Codes ;
- API Kelio pour la récupération des salariés présents ;
- système d'envoi d'e-mails intégré au portail intranet.

2.2.4 Conception de la solution

La première étape a consisté à concevoir la structure de données nécessaire au fonctionnement de l'application.

Deux modèles principaux ont été créés.

Modèle Site

Le modèle **Site** représente une implantation de l'entreprise. Il contient notamment :

- un identifiant unique de type `SlugField` utilisé dans les URLs (par exemple `saint-avertin`);
- le nom affiché du site (CharField) ;
- l'URL externe du document PDF des consignes de sécurité (URLField);
- les adresses e-mail des responsables à prévenir en cas d'évacuation (TextField adresses emails séparées par des virgules);
- les informations du responsable du site (Nom et emal).

Ce modèle permet de centraliser toutes les informations spécifiques à chaque implantation et d'en ajouter de nouveaux facilement depuis l'interface d'administration Django.

Modèle Visiteur

Le modèle **Visiteur** permet de stocker les informations relatives à une visite :

- nom (CharField) ;
- prénom (CharField) ;
- société (CharField) ;
- personne visitée (CharField) ;
- heure d'arrivée (renseignée automatiquement à la création) (DateTimeField) ;
- heure de départ (DateTimeField) ;
- signature électronique (TextField) (stockée en base64) ;
- indicateur de sortie automatique (permettant de distinguer les départs réellement enregistrés des fermetures automatiques effectuées par le job de nuit).

Chaque visiteur est relié à un site grâce à une relation `ForeignKey` avec l'option `on_delete=PROTECT`, qui empêche la suppression d'un site ayant des visiteurs associés afin de préserver l'historique des visites. Cette architecture garantit une séparation stricte des données entre les différents sites.

Une méthode `clean()` a également été ajoutée dans le formulaire afin de normaliser automatiquement la casse des champs texte saisis par les visiteurs, garantissant une cohérence des données quelle que soit la façon dont l'utilisateur a saisi son nom ou sa société.

Une fois les modèles définis, les migrations Django ont été générées puis appliquées afin de créer les tables nécessaires dans la base de données.

Génération des migrations

Lors de la génération des migrations, une difficulté technique a été rencontrée. Le projet intranet utilisant deux bases de données simultanément (SQLite en

développement et SQL Server en production via un `DATABASE_ROUTERS` configuré dans `settings.py`), Django tentait de contacter SQL Server au moment de l'exécution de la commande `makemigrations`, provoquant une erreur `pyodbc.InterfaceError` car le serveur de production n'était pas accessible depuis l'environnement de développement local. La solution retenue a été de créer manuellement le fichier de migration `0001_initial.py` dans le dossier `visiteur/migrations/`, puis d'appliquer la migration uniquement sur SQLite via la commande `migrate --database=default`, sans toucher à SQL Server.

Par la suite, j'ai constaté qu'il suffisait de commenter le routeur pour exécuter la migration ; c'est cette méthode que j'ai privilégiée pour la suite du projet.

2.2.5 Réalisation

Le développement de l'application a été réalisé progressivement en suivant les différentes fonctionnalités identifiées lors de l'analyse du besoin.

Architecture de routage multi-sites

Le cœur de l'architecture repose sur un système de routage Django basé sur un identifiant de site (`SlugField`) intégré directement dans chaque URL. Par exemple, la tablette du site de Saint-Avertin ouvre l'URL `/visiteur/saint-avertin/`, celle de Saint-Maure `/visiteur/saint-maure/`. Toutes les vues récupèrent le site correspondant via `get_object_or_404(Site, code=site_code)` et filtrent systématiquement les données sur ce site, garantissant qu'un visiteur enregistré sur un site n'apparaît jamais sur un autre.

Gestion des visiteurs présents (Annexe C.1)

La première fonctionnalité développée a été la page d'accueil de l'application. Celle-ci affiche en temps réel la liste des visiteurs présents sur le site associé à la tablette utilisée.

Le site est automatiquement identifié grâce à l'URL utilisée par la tablette. Cette approche garantit qu'un visiteur enregistré sur un site n'apparaît jamais sur un autre.

Enregistrement d'un visiteur (Annexe C.2)

J'ai ensuite développé un formulaire de type `ModelForm` permettant aux visiteurs de saisir :

- leur nom ;
- leur prénom ;
- leur société ;
- la personne qu'ils viennent rencontrer.

Le site n'est jamais sélectionné manuellement par l'utilisateur. Il est automatiquement déterminé à partir de l'URL de la tablette et associé à l'enregistrement avant la sauvegarde via `form.save(commit=False)`.

Cette approche simplifie l'utilisation tout en évitant les erreurs de saisie.

La vue gère deux comportements distincts selon la méthode HTTP : affichage du formulaire vide en GET, validation et redirection en POST.

De plus, une normalisation automatique de la casse a également été mise en place : la première lettre de chaque mot est automatiquement mise en majuscule (grâce à la méthode clean() du formulaire qui permet de nettoyer et normaliser les données), garantissant une cohérence des données quel que soit le mode de saisie du visiteur.

Consultation des consignes de sécurité (Annexe C.3)

Après l'enregistrement, le visiteur est redirigé vers une page présentant les consignes de sécurité du site concerné.

Chaque implantation pouvant posséder des règles spécifiques, les consignes affichées sont adaptées dynamiquement au site sélectionné.

Afin de permettre au visiteur de conserver ces informations sur son téléphone, un QR Code est généré automatiquement à partir de l'URL du document PDF correspondant. Cette fonctionnalité a été développée à l'aide de la bibliothèque Python **qrcode**. La génération est réalisée côté serveur via la bibliothèque Python `qrcode`, l'image étant encodée en base64 et transmise au template pour être affichée directement dans le HTML sans fichier temporaire.

Signature électronique (Annexe C.4)

Une étape de signature électronique a ensuite été intégrée au processus.

L'étape de signature utilise la bibliothèque JavaScript `signature_pad`, hébergée localement conformément aux conventions du projet (aucun CDN externe autorisé). Le visiteur signe sur un canvas tactile. Au clic sur « Valider », la signature est convertie en image via `toDataURL()`, encodée en base64, puis envoyée au serveur via un formulaire POST. La vue vérifie côté serveur la présence de la signature et stocke uniquement la partie base64 dans le champ `signature` du modèle, sans le préfixe `` qui est réintégré dynamiquement à l'affichage.

Cette signature constitue une preuve de lecture et d'acceptation des consignes de sécurité.

Gestion des départs (Annexes C.6 et C.7)

L'application permet également d'enregistrer le départ des visiteurs.

Lorsqu'un visiteur quitte les locaux, il peut sélectionner son nom dans la liste des personnes présentes puis confirmer son départ. L'heure de sortie est alors enregistrée et le visiteur disparaît automatiquement de la liste affichée.

La vue de pointage de départ est protégée par le décorateur `@require_POST`, qui empêche tout déclenchement accidentel via un simple lien GET. La récupération du visiteur s'effectue via `get_object_or_404(Visiteur, pk=pk, site__code=site_code)`, une double vérification qui empêche qu'un visiteur d'un site soit marqué sorti depuis la tablette d'un autre site.

Mode groupe (Annexe C.5)

Lorsque plusieurs visiteurs d'une même société arrivent ensemble pour rencontrer la même personne, un mode groupe permet d'accélérer l'enregistrement. Après la validation du premier visiteur, un bouton « Ajouter un collègue » redirige vers le formulaire avec la société et la personne visitée déjà pré-remplies via des paramètres GET dans l'URL. Chaque membre du groupe signe néanmoins individuellement les consignes de sécurité.

Message de confirmation (Annexe C.8)

Une fois arrivé sur la page de confirmation, si l'utilisateur clique sur terminer, il retourne à la page d'accueil et un message de succès apparaît et disparaît au bout de 10 secondes grâce à un petit script en JavaScript.

Fermeture automatique des visites

Afin d'éviter les oublis de pointage, un traitement automatique a été développé.

Chaque soir à 23 h 59, une tâche planifiée recherche les visiteurs encore présents dans le système et les marque automatiquement comme sortis. Un indicateur spécifique permet de distinguer ces sorties automatiques des départs réellement enregistrés.

Ce job planifié créé dans `core/jobs/daily/visiteur_auto_sortie.py` s'exécute chaque soir à 23h59. Il récupère tous les visiteurs dont le champ `heure_sortie` est nul, tous sites confondus, leur affecte une heure de sortie à 23h59 et passe le champ `sortie_auto` à `True`. Lors du débogage de ce job, une difficulté supplémentaire a été rencontrée : un fichier de verrou (lock) généré par le système empêchait parfois son exécution. La suppression manuelle de ce fichier a permis de rétablir le bon fonctionnement du traitement automatique.

Gestion des évacuations (Annexes C.9 et C.10)

Une page dédiée permet de consulter instantanément la liste des visiteurs présents sur un site donné.

Elle récupère également les informations de présence des salariés via l'API Kelio, en ne conservant que les personnes dont le dernier badgeage est une entrée.

L'application a été conçue pour rester fonctionnelle même en cas d'indisponibilité temporaire de Kelio. Dans ce cas, les visiteurs restent visibles et un message informe l'utilisateur que les données salariés sont momentanément indisponibles.

Cette page d'évacuation utilise une fonction que j'ai créée dans la vue : recuperer_salaries_visiteurs_presents(site) qui récupère tous les salariés et visiteurs présents. Cette fonction est utilisé dans plusieurs autres fonctions et dans certains templates. (par exemple : evacuation.html).

Deux templates partiels ont également été créés, l'un pour la liste des visiteurs, l'autre pour la liste des salariés, inclus via {% include %} dans les pages concernées afin d'éviter la duplication du HTML.

Envoi des listes d'évacuation

Une fonctionnalité d'envoi d'e-mails a également été développée.

Depuis la page d'évacuation, il est possible de transmettre automatiquement aux responsables du site la liste complète des personnes présentes. Les destinataires sont configurés directement dans les paramètres du site et le contenu du message est généré dynamiquement (grâce à la vue et un template dédié).

Historique des visites (Annexes C.11 et C.12)

Une page d'historique a été développée à destination des administrateurs, accessible uniquement aux utilisateurs connectés à l'intranet. Elle liste l'ensemble des visites enregistrées sur tous les sites et propose plusieurs filtres transmis en paramètres GET : par site, par date, par société, par hôte et par statut (présent, sorti, sorti automatiquement).

Chaque ligne du tableau est cliquable et redirige vers **une page de détail (Annexes 13 et 14)** affichant l'ensemble des informations de la visite : coordonnées du visiteur, heure d'arrivée et de départ, signature électronique, et indication si la sortie a été enregistrée automatiquement par le job de nuit.

Un soin particulier a été apporté à la navigation : lorsqu'un administrateur consulte le détail d'une visite depuis l'historique filtré, le bouton « Retour à l'historique » restaure automatiquement les filtres actifs en les encodant dans l'URL, évitant ainsi d'avoir à les ressaisir manuellement.

Interface tablette

Une feuille de style dédiée `tablette.css` surcharge les styles Bootstrap pour imposer des tailles adaptées à une utilisation tactile : corps de texte à 24px minimum, boutons d'au moins 80px de hauteur avec un espacement généreux pour éviter les clics accidentels, champs de saisie à 70px de hauteur. Des attributs HTML tactiles ont été ajoutés sur les champs de saisie : `autocorrect="off"` pour désactiver la correction automatique sur les noms et sociétés, `autocapitalize="words"` pour capitaliser automatiquement la première lettre de chaque mot. La page d'accueil se recharge automatiquement après 10 secondes lorsqu'un message de bienvenue ou d'au revoir est affiché, afin d'éviter toute

confusion pour le visiteur suivant. L'interface a été testée via Chrome DevTools en mode responsive avant d'être validée sur la tablette Android réelle.

2.2.6 Difficultés rencontrées

Gestion multi-sites

La principale difficulté du projet concernait la gestion simultanée de plusieurs sites au sein d'une même application.

Pour répondre à ce besoin, j'ai conçu l'application autour du modèle **Site** relié aux visiteurs par une clé étrangère. Chaque tablette utilise une URL spécifique contenant l'identifiant du site concerné.

Cette architecture garantit une isolation complète des données entre les différentes implantations de l'entreprise.

Déploiement Linux

Lors des premiers tests de déploiement sur un serveur Linux, certaines ressources statiques ne s'affichaient plus correctement.

Après analyse, j'ai identifié une différence de comportement entre Windows et Linux concernant la sensibilité à la casse des noms de fichiers. Certains chemins référencés dans le projet ne correspondaient pas exactement au nom réel des fichiers.

La correction des noms concernés a permis de résoudre définitivement le problème.

Gestion des données incomplètes

Au cours du développement, j'ai identifié une limite dans le processus d'enregistrement. Le visiteur est actuellement créé dans la base de données avant la validation de sa signature.

Si l'utilisateur abandonne le processus avant cette étape, une visite incomplète reste enregistrée. Une amélioration envisagée consiste à stocker temporairement les informations en session puis à créer l'enregistrement uniquement après validation complète du parcours.

Soumissions multiples sur tablette

Lors des tests sur tablette Android, il a été constaté qu'un appui rapide et répété sur les boutons de validation pouvait provoquer l'envoi de plusieurs requêtes simultanées. Ce comportement entraînait la création de doublons en base de données ainsi qu'une désynchronisation de la signature électronique, seul le dernier enregistrement conservant la signature. Ce bug a été documenté dans le fichier de documentation de l'application pour être traité dans une version ultérieure.

2.2.7 Tests réalisés

Afin de garantir la fiabilité de l'application, plusieurs tests unitaires et fonctionnels ont été réalisés.

Les tests développés permettent notamment de vérifier :

- la création des sites ;
- la création des visiteurs ;
- le bon fonctionnement du pointage de sortie ;
- l'exécution du traitement automatique quotidien ;
- l'isolation des données entre les différents sites ;
- le comportement de la page d'évacuation avec des données Kelio simulées (Les tests de la page d'évacuation utilisent unittest.mock.patch pour simuler les réponses de l'API Kelio sans dépendre du système réel, permettant de vérifier que seuls les salariés dont le dernier badgeage est une entrée sont affichés.) ;
- l'impossibilité de valider une visite sans signature électronique.

Des tests complémentaires ont également été réalisés directement sur tablette Android afin de valider l'ergonomie et le bon déroulement du workflow complet d'enregistrement.

2.2.8 Résultats obtenus (Annexes C.15 et C.16)

À l'issue du développement, l'application permet :

- l'enregistrement numérique des visiteurs ;
- la consultation en temps réel des visiteurs présents ;
- l'affichage et l'acceptation des consignes de sécurité ;
- la signature électronique des visiteurs ;
- la gestion des départs ;
- la fermeture automatique des visites oubliées ;
- la génération de listes d'évacuation ;
- l'envoi automatique des informations aux responsables concernés ;
- l'isolation complète des visiteurs entre les différents sites de l'entreprise.

Cette mission m'a permis d'approfondir mes compétences en développement web avec Django, en conception de bases de données relationnelles, en intégration d'API, en génération dynamique de documents et en mise en œuvre de tests unitaires.

2.3 Mission 3 : Développement de l'application de gestion des congés

2.3.1 Contexte et besoin

Dans le cadre de l'évolution de l'intranet de SECTRONIC, un besoin de modernisation du processus de gestion des absences a été identifié.

Les demandes de congés et d'absences étaient jusqu'alors traitées à travers différents échanges entre les salariés, les responsables hiérarchiques, le secrétariat et la direction générale. Cette organisation nécessitait de nombreuses interventions manuelles et rendait le suivi des demandes plus complexe. L'ensemble du processus reposait sur un formulaire papier, ce qui rendait le suivi des demandes difficile à centraliser.

Les collaborateurs ne disposaient pas d'un outil unique leur permettant de consulter facilement l'état de leurs demandes. De leur côté, les responsables devaient suivre les validations à travers différents échanges, sans disposer d'une vision centralisée des absences en cours.

Afin de répondre à ces problématiques, il a été décidé de développer une application web intégrée à l'intranet permettant de dématérialiser entièrement le processus de gestion des congés.

L'objectif principal de cette mission était donc de concevoir une application capable de gérer le cycle complet d'une demande d'absence, depuis sa création jusqu'à sa validation finale, tout en assurant la traçabilité des décisions prises et l'automatisation des notifications.

2.3.2 Analyse du besoin

Une phase d'analyse a été réalisée afin d'identifier les différents acteurs intervenant dans le processus de validation.

Quatre profils principaux ont été définis :

- les salariés ;
- les chefs de service ;
- le secrétariat ;
- la direction générale.

Le processus métier existant a permis d'identifier plusieurs besoins fonctionnels :

- création d'une demande d'absence ;
- suivi de l'état d'avancement de la demande ;
- validation hiérarchique en plusieurs étapes ;
- gestion des refus avec justification obligatoire ;
- conservation d'un historique des validations ;

- génération automatique de documents justificatifs ;
- envoi de notifications électroniques ;
- consultation des absences validées sous forme de calendrier.

L'application devait également permettre de conserver une trace de chaque décision afin de garantir la traçabilité complète du processus.

2.3.3 Technologies utilisées

Le développement de l'application a été réalisé à l'aide des technologies suivantes :

- Python 3.12 ;
- Framework Django ;
- HTML5 ;
- CSS3 ;
- Bootstrap 5 ;
- JavaScript ;
- SQLite pour l'environnement de développement ;
- GitLab pour le versionnement ;
- FullCalendar pour l'affichage du planning des absences ;
- Génération PDF (pymupdf) ;
- Module interne `core.emailing` pour l'envoi des courriers électroniques.

L'application a été développée de manière à pouvoir être migrée ultérieurement vers SQL Server lors de son déploiement en production.

2.3.4 Conception de la solution

La conception de l'application repose sur un modèle principal nommé **DemandeAbsence**.

Ce modèle centralise l'ensemble des informations relatives à une demande d'absence ainsi que son cycle complet de validation.

Les informations enregistrées comprennent :

Informations du demandeur

- nom (CharField) ;
- prénom (CharField) ;
- adresse électronique (EmailField) .

Période d'absence

- date de début (DateField) ;
- date de fin (DateField) ;

- heure de début (TimeField) ;
- heure de fin (TimeField) .

Cette structure permet de gérer à la fois les absences sur plusieurs jours et les absences de courte durée sur une même journée.

Types d'absence

Plusieurs catégories d'absences ont été intégrées :

- congés payés ;
- RTT ;
- jour d'ancienneté ;
- sans solde ;
- modulation ;
- compteur de souplesse ;
- récupération ;
- événement familial ;
- autres motifs.

Des champs complémentaires permettent de stocker les informations spécifiques à certains types d'absence, comme les justificatifs ou les dates de récupération.

Gestion des validations

Afin d'assurer la traçabilité du processus, le modèle conserve les informations relatives aux différents validateurs :

- identité du chef de service (nom, prenom) (CharField) ;
- identité du secrétariat (CharField) ;
- identité de la direction générale (CharField) ;
    - dates de validation (chef_service, secretariat, dg) (DateTimeField) ;
- motifs de refus éventuels (chef_service, secretariat, dg) (TextField) .

Cette approche permet de reconstituer l'historique complet de chaque demande.

2.3.5 Développement de l'application

Le développement a été réalisé de manière progressive en suivant les différentes étapes du processus métier. Avant de commencer, j'ai analysé le formulaire papier existant afin d'en extraire les règles métier essentielles, puis rédigé un cahier des charges décrivant les cas d'usage pour chaque profil utilisateur. Ce document a été validé par mon maître de stage avant le début du développement.

Conception du modèle de données

Le modèle principal `DemandeAbsence` centralise l'ensemble des informations relatives à une demande et à son cycle complet de validation. Une réflexion préalable a été menée sur la gestion des sessions utilisateur : l'intranet ne disposant pas encore de système d'authentification au moment du développement, il était impossible de relier automatiquement une demande à un compte utilisateur identifié. Le modèle a donc été conçu pour fonctionner sans session dans cette première version, tout en anticipant l'évolution future : les champs liés aux équipes et aux utilisateurs ont été laissés en commentaire dans le code, prêts à être activés lorsque le système d'authentification sera disponible.

Les neuf types d'absence ont été définis comme des choix (`choices`) Django : congés payés, RTT, jour d'ancienneté, sans solde, modulation, compteur de souplesse, récupération, événement familial et autres motifs. Des champs complémentaires conditionnels permettent de stocker les informations spécifiques à certains types, comme le justificatif pour un événement familial ou la date de récupération.

Pour chaque niveau de validation (chef de service, secrétariat, direction générale), le modèle conserve l'identité du validateur, la date de validation et le motif de refus éventuel, garantissant une traçabilité complète du circuit.

Création d’une demande (Annexes D.4 et D.5)

La première fonctionnalité développée concerne la soumission d’une demande d’absence.

Le formulaire de soumission est un `ModelForm` Django qui adapte dynamiquement l'affichage de certains champs selon le type d'absence sélectionné, grâce à un script JavaScript. Par exemple, le champ de justificatif n'apparaît que si le type « événement familial » est sélectionné.

Le salarié renseigne :

- ses informations personnelles ;
- les dates concernées ;
- le type d’absence ;
- les éventuelles informations complémentaires.

Lors de la validation du formulaire :

- la demande est enregistrée en base de données ;
- son statut passe à « En attente Chef de service » ;
- une notification est automatiquement envoyée au responsable concerné.

Une fenêtre de confirmation Bootstrap (modal) s'ouvre lorsque l'employé clique sur « Soumettre ma demande ». Ce modal affiche un récapitulatif de la demande (nom, prénom, dates, type d'absence et durée calculée en jours ouvrés) avant l'envoi définitif. La logique est entièrement gérée en JavaScript : au clic sur le bouton, les valeurs du formulaire sont lues et injectées dans les éléments du modal avant son ouverture.

Cette fenêtres de confirmation a été ajoutée avant l'envoi définitif afin de limiter les erreurs de saisie.

Annulation d'une demande (Annexes D.6)

Un salarié peut annuler une demande encore en attente de validation par le chef de service. Un bouton de suppression apparaît sur ces demandes depuis la page d'accueil, accompagné d'un modal de confirmation affichant les informations de la demande concernée. Pour éviter que le clic sur ce bouton ne déclenche aussi la navigation vers la page de détail (la ligne entière étant cliquable), la propagation de l'événement JavaScript a été bloquée via `event.stopPropagation()`.

Workflow de validation (Annexes D.1, D.2, D.3, D.7, D.8, D.9, D.10, D.11, D.12)

Le cœur de l'application repose sur un système de validation hiérarchique en plusieurs étapes.

Chaque demande suit automatiquement le circuit suivant :

Salarié → Chef de service → Secrétariat → Direction Générale

À chaque étape, la vue vérifie le statut courant de la demande, enregistre les informations du validateur, met à jour le statut via les cinq valeurs définies dans le modèle (`en_attente_chef`, `en_attente_secretariat`, `en_attente_dg`, `approuve`, `refuse`), puis déclenche les notifications appropriées.

À chaque étape, le validateur peut :

- approuver la demande ;
- refuser la demande.

En cas de refus :

- la saisie d'un motif devient obligatoire ;
- le statut passe à « Refusé » ;
- le demandeur reçoit automatiquement une notification.

En cas d'approbation :

- le statut est mis à jour ;
- la demande est transmise automatiquement à l'étape suivante.

Ce mécanisme permet d'automatiser entièrement le processus de validation.

Chaque tableau de bord dispose de trois onglets Bootstrap : les demandes en cours, les demandes approuvées et les demandes refusées. Pour le chef de service et le secrétariat, un surlignage coloré signale visuellement les demandes approuvées à leur niveau mais refusées par un niveau supérieur (en rouge) ou encore en attente du niveau suivant (en orange). Pour ça on récupère dans la vue toutes les demandes traitées et on regarde où elles en sont et ont transmet ça dans le template.

Page de détail d'une demande (Annexes D.13, D.14, D.17, D.18)

Une page de détail est accessible depuis n'importe quel tableau de bord en cliquant sur une ligne. Elle affiche en lecture seule l'ensemble des informations de la demande et s'adapte automatiquement selon son statut :

- Si la demande est **en cours de validation**, le circuit de validation est affiché avec l'étape en cours clairement identifiée, permettant au salarié de savoir précisément où en est sa demande.
- Si la demande a été **refusée**, la page indique quel niveau hiérarchique a refusé (chef de service, secrétariat ou direction générale), affiche le motif de refus saisi par le validateur, et propose le téléchargement du coupon PDF de refus.
- Si la demande a été **approuvée**, la page confirme la validation complète du circuit et propose le téléchargement du coupon PDF d'autorisation.

Cette page centralise ainsi toutes les informations relatives à une demande et permet à chaque acteur du processus de disposer d'une vision complète de son historique de validation. On récupère et filtre tout les éléments dna sla vue qu'on envoie ensuite dans le template.

Gestion des notifications (Annexes D.18 et D.19)

Afin de réduire les interventions manuelles, un système complet de notifications automatiques a été développé.

Les courriers électroniques sont envoyés lors :

- de la création d'une demande ;
- de la validation par le chef de service ;
- de la validation par le secrétariat ;
- de la validation finale par la direction générale ;
- d'un refus à n'importe quelle étape.

Les courriers électroniques sont envoyés via le module interne `core/emailing.py` déjà présent dans le projet intranet. À chaque étape du circuit, un email HTML est généré via un template dédié et envoyé automatiquement à la personne concernée. Le circuit de transmission des emails fonctionne ainsi : l'employé saisit l'email de son chef dans le formulaire de demande, le chef saisit l'email du secrétariat lors de sa validation, le secrétariat saisit l'email de la DG, et la DG envoie directement à l'email de l'employé stocké dans la demande.

Génération automatique des documents PDF

Une fonctionnalité importante de l'application concerne la génération automatique de documents PDF.

Deux types de documents sont produits :

Coupon d'autorisation (Annexe D.20)

Lorsque la demande est approuvée par la direction générale :

- un document PDF est généré automatiquement ;
- celui-ci est envoyé au salarié par courrier électronique ;
- il reste disponible au téléchargement depuis l'application.

Coupon de refus (Annexe D.21)

Lorsqu'une demande est refusée :

- un document PDF de refus est généré ;
- le motif du refus est intégré au document ;
- le fichier est transmis automatiquement au salarié.

Cette fonctionnalité permet de conserver une trace officielle des décisions prises.

La génération des coupons PDF utilise la bibliothèque `pymupdf` (également appelée `fitz`) déjà présente dans le projet. Deux types de documents sont produits depuis un fichier `pdf.py` dédié. Le coupon d'autorisation est généré lors de la validation finale par la DG, avec une mise en page épurée. Le coupon de refus identifie automatiquement le niveau ayant refusé la demande, affiche le motif de refus avec un bandeau rouge, et est généré quelle que soit l'étape du circuit où le refus intervient.

Calcul automatique des durées d'absence

Un fichier `utils.py` centralise le calcul de la durée côté serveur, conformément aux bonnes pratiques Django qui consistent à séparer la logique métier des vues. La fonction `calculer_duree_absence` gère deux cas distincts : pour les absences sur une seule journée avec des horaires renseignés, elle calcule la durée en heures et minutes avec un affichage soigné (par exemple « 1h15 » plutôt que « 1,25 heure(s) »). Pour les absences sur plusieurs jours, elle compte les jours ouvrés du lundi au vendredi en excluant les weekends. Cette fonction est importée dans toutes les vues de détail et de validation, et la durée calculée est transmise au template via le contexte.

En parallèle, un calcul en temps réel a été intégré côté client en JavaScript : lorsque l'employé sélectionne ses dates dans le formulaire, le nombre de jours ouvrés s'affiche instantanément, lui permettant de vérifier la durée de son absence avant de

soumettre. Ce script JavaScript a été extrait dans un fichier statique dédié `calculerJours.js`, réutilisable dans plusieurs pages sans duplication.

Cette information est affichée dans les interfaces de consultation et de validation.

Vérification des chevauchements de dates

Une vérification des chevauchements a été ajoutée dans la méthode clean() du formulaire Django. Si un salarié tente de soumettre une demande dont les dates recoupent une demande déjà approuvée ou en cours de validation à son nom (identifié par son nom, prénom et email) le formulaire affiche un message d'erreur et refuse la soumission. La détection repose sur un filtre ORM Django comparant les dates de début et de fin des demandes existantes.

Pour cela, on récupère toutes les demandes d'absences de la personne grâce à DemandeAbsence.objects.filter(nom=cleaned_data.get('nom'),prenom=cleaned_data.get('prenom'),email=email,statut__in=['approuve','en_attente_chef','en_attente_secretariat', 'en_attente_dg']), Puis on vérifie si les dates se chevauchent avec : date_debut__lte=date_fin, date_fin__gte=date_debut.

Sécurisation des formulaires contre les doubles soumissions

Lors des premiers tests, il a été constaté qu'un utilisateur pouvait cliquer plusieurs fois sur le bouton de validation avant la fin du traitement, ce qui pouvait provoquer la création de doublons en base de données et l'envoi multiple de notifications. Plusieurs approches JavaScript ont été étudiées, notamment la désactivation du bouton après le premier clic, mais elles se sont révélées insuffisantes dans le contexte de l'application en raison de l'utilisation du modal Bootstrap de confirmation qui s'intercale entre le clic et la soumission réelle. La solution retenue consiste à bloquer toute interaction avec la page au moment de la confirmation via la propriété CSS `pointerEvents`, appliquée à l'ensemble de la page dès le clic sur le bouton de validation dans le modal.

Cette protection empêche efficacement les doubles soumissions.

Calendrier des absences (Annexe D.15)

Un calendrier interactif a été intégré dans les tableaux de bord du chef de service, du secrétariat et de la direction générale à l'aide de la bibliothèque FullCalendar. Il affiche toutes les absences approuvées sous forme d'événements colorés et propose trois vues : par mois, par semaine et en liste. Chaque événement est cliquable et redirige vers la page de détail de la demande concernée.

Une difficulté technique a été rencontrée lors de la transmission des données Python vers JavaScript. La solution habituelle consistant à stocker du JSON dans un attribut HTML posait un problème : Django échappe automatiquement les guillemets et apostrophes pour éviter les failles de sécurité XSS, ce qui générait des entités HTML qui cassaient le JSON. La solution retenue utilise une balise `<script`

`type="application/json">` dont le contenu n'est pas échappé par Django et n'est pas exécuté par le navigateur. Le contenu est ensuite lu en JavaScript via `JSON.parse()`. C'est la méthode recommandée pour transmettre des données Python vers JavaScript dans Django.

Une difficulté supplémentaire concernait l'initialisation du calendrier : celui-ci ne se dessinait pas correctement lorsque son onglet était masqué au chargement de la page, car les dimensions de l'élément n'étaient pas encore calculées. Ce problème a été résolu en initialisant FullCalendar sur l'événement `shown.bs.tab` de Bootstrap, qui se déclenche uniquement lorsque l'onglet devient visible.

Pour éviter de dupliquer le code du calendrier, un template partiel `_calendrier.html` a été créé et inclus via `{% include %}` dans les tableaux du secrétariat et de la DG. La directive `{{ block.super }}` a été utilisée dans les blocs Django pour ne pas écraser le contenu existant lors de l'ajout de ressources CSS et JavaScript supplémentaires.

2.3.6 Difficultés rencontrées

Gestion du workflow de validation

La principale difficulté du projet concernait la mise en place du cycle de validation à plusieurs niveaux.

Chaque étape devait :

- vérifier le statut courant de la demande ;
- enregistrer les informations du validateur ;
- mettre à jour le statut ;
- déclencher les notifications appropriées.

Une attention particulière a été portée à la cohérence des transitions entre les différents états afin d'éviter toute incohérence dans le traitement des demandes.

2.3.7 Tests réalisés

Plusieurs phases de tests ont été réalisées tout au long du développement.

Les tests ont notamment porté sur :

- la création des demandes ;
- les validations successives ;
- les refus avec motif obligatoire ;
- la génération des documents PDF ;
- l'envoi des notifications électroniques ;
- le calcul des durées d'absence ;

- l'annulation d'une demande ;
- le calendrier FullCalendar ;
- la protection contre les doubles soumissions.

Ces vérifications ont permis de valider le bon fonctionnement de l'ensemble du processus métier.

2.3.8 Perspectives d'évolution (Annexe D.24)

Plusieurs améliorations ont été identifiées pour les futures versions de l'application.

La principale évolution concerne **l'intégration du système d'authentification** de l'intranet.

Cette évolution permettra :

- l'identification automatique des utilisateurs ;
- l'attribution des rôles (employé, chef de service, secrétariat, direction générale) ;
- la sécurisation des accès ;
- le filtrage des données selon l'utilisateur connecté ;
- l'association automatique des responsables hiérarchiques ;
- la restriction des tableaux de bord aux équipes concernées.

Afin d'anticiper cette évolution, j'ai rédigé un document technique structuré en neuf étapes numérotées, détaillant précisément l'ensemble des modifications à apporter fichier par fichier (models.py, views.py, forms.py, templates, settings.py) ainsi que les tests à réaliser à chaque étape pour valider le bon fonctionnement des restrictions d'accès. Ce document permettra à n'importe quel développeur de reprendre l'intégration de l'authentification sans avoir à analyser l'application depuis le début.

D'autres améliorations sont également envisagées :

- notifications internes dans l'intranet ;
- statistiques sur les absences ;
- tableaux de bord décisionnels ;
- filtrage avancé du calendrier par équipe ;

2.3.9 Bilan de la mission (Annexes D.22 et D.23)

Cette mission m'a permis de participer à la conception et au développement d'une application métier complète intégrée à l'intranet de l'entreprise.

J'ai approfondi mes compétences en analyse de situation pour concevoir un cahier des charges, en développement web avec Django, en conception de bases de données, en automatisation de processus métier, en génération de documents PDF et en intégration de services de notification.

Le projet a abouti à une solution fonctionnelle permettant de dématérialiser entièrement la gestion des demandes d'absence tout en apportant davantage de traçabilité, de visibilité et d'automatisation aux différents acteurs impliqués dans le processus.

2.4 Mission 4 : Tests de l'application M-EXTEND

2.4.1 Contexte et besoin

Dans le cadre du développement d'un nouveau banc de test nommé **M-EXTEND**, un besoin de validation fonctionnelle est apparu avant son déploiement et son utilisation en production.

Ce projet, développé par un collaborateur du Bureau d'Études, a pour objectif de réaliser des mesures de tension et de courant à l'aide d'un manche équipé de plusieurs boutons. Avant sa mise en service, il était nécessaire de valider le bon fonctionnement de l'ensemble du banc.

Avant la mise en service de cette solution, il était nécessaire de vérifier le bon fonctionnement de l'application, de s'assurer de la cohérence des données enregistrées et d'identifier d'éventuelles anomalies pouvant affecter les utilisateurs.

J'ai ainsi été chargé de réaliser une campagne complète de tests fonctionnels sur l'application M-EXTEND et sur son interface intranet associée.

2.4.2 Analyse du fonctionnement

Avant de débuter les tests, une phase de découverte du projet a été réalisée avec le développeur de l'application.

Cette présentation m'a permis de comprendre :

- le fonctionnement général du banc de test ;
- les différentes étapes exécutées lors d'un contrôle ;
- les mesures réalisées par le système ;
- les informations enregistrées dans la base de données ;
- les données affichées dans l'intranet ;
- le mécanisme d'export PDF des résultats.

Le déroulement d'un test suit plusieurs étapes successives durant lesquelles l'opérateur doit appuyer sur différents boutons présents sur le manche du banc afin de déclencher des mesures de courant et de tension.

Les résultats sont ensuite stockés puis affichés dans une interface intranet permettant de consulter l'historique des cartes testées.

L'objectif principal de ma mission consistait à vérifier que chaque fonctionnalité se comportait conformément aux attentes et à identifier les éventuels dysfonctionnements.

2.4.3 Méthodologie de test

Afin de réaliser des tests les plus complets possibles, j'ai adopté une approche consistant à reproduire différents scénarios d'utilisation réels ainsi que des situations inhabituelles susceptibles de provoquer des erreurs.

Les vérifications ont porté sur plusieurs aspects :

- la gestion des informations de session ;
- la validation des champs de saisie ;
- le comportement des boutons du banc ;
- la robustesse de l'application face à des actions inattendues ;
- l'affichage des informations dans l'intranet ;
- la cohérence des données exportées au format PDF.

Pour chaque anomalie détectée, une description détaillée a été rédigée afin de faciliter l'analyse et la correction par le développeur.

Les anomalies identifiées ont été classées selon leur niveau de sévérité afin de prioriser les corrections à apporter avant la mise en production.

2.4.4 Tests réalisés

Gestion de la session utilisateur

Les premiers tests ont porté sur les informations saisies avant le lancement d'un test.

J'ai constaté qu'il était possible de démarrer une séquence sans renseigner le nom de l'opérateur.

J'ai également observé que les champs « Opérateur » et « LCT » (Lancement) pouvaient être modifiés ou supprimés alors qu'un test était déjà en cours, sans qu'aucun message d'erreur ne soit affiché.

Enfin, aucune validation n'était appliquée sur ces champs. Il était possible de saisir des chaînes de caractères très longues ou des caractères spéciaux sans restriction particulière.

Ces comportements peuvent conduire à l'enregistrement d'informations incohérentes ou inexploitables dans l'intranet.

Vérification du comportement des boutons

Une partie importante de la campagne de tests a concerné les boutons du manche utilisés lors des différentes étapes de la séquence.

Plusieurs anomalies ont été identifiées.

Lors du test du bouton bleu, un appui simultané sur plusieurs boutons validait malgré tout l'étape alors qu'un appui unique sur le bouton demandé était attendu.

Un comportement similaire a été observé pour le bouton vert lorsqu'il était actionné en même temps que le bouton noir.

J'ai également constaté qu'un maintien des boutons dès le lancement du test permettait de contourner certaines étapes de validation.

Dans certains cas, la première mesure était validée avec une valeur incorrecte qui était ensuite enregistrée dans l'intranet comme un résultat valide.

Ces comportements peuvent fausser les résultats des essais et compromettre la fiabilité des mesures enregistrées.

Validation de la préparation du banc

Les tests ont également porté sur les informations saisies lors de la préparation du banc.

Plusieurs défauts de validation ont été observés :

- les champs DRF acceptaient n'importe quel type de saisie ;
- aucune limite de longueur n'était appliquée ;
- certaines valeurs incorrectes provoquaient un arrêt complet de l'application.

Par exemple, la saisie d'une variante différente de celle attendue ou d'une température hors plage entraînait une erreur bloquante interrompant totalement le processus de test.

L'application ne proposait aucun mécanisme de récupération et devait être redémarrée manuellement.

Cette situation a été identifiée comme l'une des anomalies les plus critiques de la campagne de tests.

Vérification de l'interface utilisateur

L'analyse de l'interface graphique a permis de détecter plusieurs défauts d'affichage.

Le titre « Préparation du banc de test » était partiellement coupé dans certaines fenêtres.

Lors de l'affichage de la séquence de tests, certaines lettres étaient légèrement masquées.

Bien que ces anomalies n'affectent pas le fonctionnement du logiciel, elles dégradent l'expérience utilisateur et peuvent nuire à la lisibilité de certaines informations.

Vérification de la séquence de test

Lors de l'exécution complète d'un cycle de test, plusieurs étapes défilaient trop rapidement pour être correctement lues par l'opérateur.

Les étapes 19, 21 et 28 étaient particulièrement concernées.

Le temps d'affichage insuffisant ne permettait pas à l'utilisateur de prendre connaissance des instructions affichées à l'écran.

2.4.5 Contrôle de l'intégration dans l'intranet

Une fois les tests du banc terminés, j'ai vérifié les données enregistrées dans l'intranet.

Plusieurs incohérences ont été détectées.

La colonne « Variante » affichait systématiquement une valeur vide malgré la saisie d'une variante lors de la préparation.

La durée réelle des tests n'était également jamais affichée.

En consultant le détail des étapes réalisées, certaines lignes ne contenaient aucune valeur mesurée alors que les étapes correspondantes avaient été exécutées avec succès.

J'ai également constaté que les informations saisies lors de la préparation du banc, telles que :

- le numéro DRF ;
- l'indice ;
- la température ;
- l'hygrométrie ;

n'étaient pas accessibles depuis l'interface intranet.

Cette absence limite fortement la traçabilité des essais réalisés.

2.4.6 Vérification de l'export PDF

L'application permet d'exporter les résultats d'un test au format PDF.

Plusieurs contrôles ont été effectués sur ces documents.

J'ai constaté que certaines informations importantes n'étaient pas présentes dans les rapports générés :

- la variante utilisée ;
- la durée du test ;
- le nombre d'étapes validées avec succès.

Par ailleurs, certains éléments graphiques dépassaient du cadre prévu dans le document, notamment dans la section présentant la séquence principale de test.

Ces anomalies peuvent compliquer l'exploitation et l'archivage des résultats.

2.4.7 Rédaction du rapport d'anomalies

À l'issue de la campagne de tests, j'ai réalisé une synthèse complète des anomalies détectées.

Un document a été transmis au développeur :

- un rapport structuré synthétisant les anomalies par catégorie avec leur description, leur impact potentiel et les conditions permettant de les reproduire.

Cette documentation a permis de fournir une vision claire des points à corriger avant la mise en production de l'application.

2.4.8 Résultats obtenus

Cette mission a permis d'identifier plusieurs anomalies fonctionnelles, ergonomiques et techniques susceptibles d'affecter la fiabilité du banc de test et la qualité des données enregistrées.

Les principaux problèmes détectés concernaient :

- les validations insuffisantes des champs de saisie ;
- le comportement incohérent de certains boutons ;
- les blocages complets de l'application lors de certaines erreurs ;
- les informations manquantes dans l'intranet ;
- les données absentes dans les exports PDF.

Les rapports produits ont servi de base de travail au développeur afin de corriger ces anomalies et d'améliorer la robustesse globale de l'application.

Cette mission m'a permis de découvrir une démarche complète de recette fonctionnelle et de validation logicielle. J'ai également développé mes compétences en analyse de comportement applicatif, en identification d'anomalies et en rédaction de rapports de tests destinés aux équipes de développement.

3. Bilan du stage

3.1 Les objectifs

L'ensemble des objectifs fixés au cours du stage ont été atteints. Les deux applications Django développées (Visiteur et Congés) sont fonctionnelles et ont été intégrées à la branche principale du dépôt GitLab de l'entreprise. La mission d'inventaire a produit les documents attendus. Les tests M-EXTEND ont été livrés dans les délais.

Deux points n'ont pas pu être finalisés dans le temps imparti : la migration des bases de données SQLite vers SQL Server, et l'intégration du système d'authentification dans l'application Congés. Ces évolutions ont été anticipées et documentées afin de faciliter leur reprise ultérieure.

3.2 Compétences techniques acquises

Ce stage m'a permis de mettre en pratique les connaissances acquises durant ma formation tout en découvrant de nouvelles technologies et méthodes de travail utilisées en entreprise.

La majeure partie de mes missions a été réalisée à l'aide du framework Django. J'ai ainsi pu approfondir mes compétences dans le développement d'applications web en participant à la conception et à la réalisation de plusieurs projets intégrés à l'intranet de l'entreprise.

J'ai notamment renforcé mes connaissances concernant :

- la programmation en Python ;
- le framework Django ;
- la création de modèles de données ;
- la gestion des migrations de bases de données ;
- le développement de vues et de formulaires ;
- la création de templates HTML ;
- l'utilisation de Bootstrap pour la conception d'interfaces responsives ;
- le développement de fonctionnalités en JavaScript ;
- la génération automatique de documents PDF ;
- l'envoi automatisé d'emails ;
- l'intégration de bibliothèques tierces ;
- l'utilisation d'API externes ;
- la réalisation de tests unitaires et fonctionnels.

Le développement de l'application « Visiteur » m'a notamment permis de découvrir la gestion d'une architecture multi-sites, la génération de QR Codes ainsi que l'intégration d'une signature électronique dans une application web.

La réalisation de l'application « Congés » m'a permis d'approfondir la mise en place de workflows complexes de validation, la gestion de statuts métier, l'automatisation des notifications et la génération dynamique de documents PDF.

Enfin, les tests réalisés sur le projet M-EXTEND m'ont permis de découvrir les méthodes de validation fonctionnelle d'une application et d'acquérir une première expérience dans la rédaction de rapports d'anomalies destinés aux développeurs.

3.3 Compétences professionnelles développées

Au-delà des aspects techniques, ce stage m'a permis de développer plusieurs compétences professionnelles essentielles.

J'ai appris à travailler dans un environnement professionnel en respectant les méthodes et les contraintes de l'entreprise. Les différents projets réalisés m'ont amené à échanger régulièrement avec mon tuteur ainsi qu'avec les autres membres du Bureau d'Études afin de comprendre les besoins exprimés et de proposer des solutions adaptées.

J'ai également développé mon autonomie. Après une phase d'apprentissage des outils et de l'environnement de travail, j'ai progressivement été capable de réaliser certaines fonctionnalités de manière indépendante, depuis l'analyse du besoin jusqu'aux phases de tests.

La rédaction de documentations techniques et de rapports de tests m'a également permis d'améliorer mes capacités de communication écrite et ma rigueur dans la formalisation des informations.

Ce stage m'a aussi sensibilisé à l'importance de la qualité logicielle, de la maintenance des applications et de la prise en compte des besoins réels des utilisateurs dans le développement d'une solution informatique.

Ce stage m'a également permis de m'intégrer à une équipe professionnelle et de prendre part à ses habitudes de travail : réunions hebdomadaires du lundi, échanges quotidiens via Mattermost, participation à la réunion trimestrielle de l'entreprise. Ces expériences m'ont appris à m'adapter au rythme et aux attentes d'un environnement professionnel réel.

3.4 Difficultés rencontrées

Comme toute expérience professionnelle, ce stage a présenté plusieurs défis techniques et organisationnels.

L'une des premières difficultés a été la découverte de l'environnement Django utilisé au sein de l'entreprise. Bien que certaines notions aient été abordées durant ma formation, j'ai dû approfondir rapidement mes connaissances afin de comprendre l'architecture des applications existantes et de pouvoir développer de nouvelles fonctionnalités.

La compréhension du fonctionnement de l'intranet et des différents outils déjà en place a également nécessité une phase d'adaptation. Plusieurs applications devaient s'intégrer dans un environnement existant tout en respectant les contraintes techniques de l'entreprise.

Lors du développement de l'application « Visiteur », la gestion multi-sites a constitué un point important de réflexion afin de garantir une séparation complète des données entre les différentes implantations de l'entreprise.

Pour l'application « Congés », la mise en œuvre du workflow de validation a demandé une attention particulière afin d'assurer le bon enchaînement des différentes étapes de validation et la cohérence des notifications envoyées aux utilisateurs.

Enfin, les campagnes de tests réalisées sur l'application M-EXTEND ont mis en évidence l'importance de prévoir de nombreux cas d'utilisation afin de détecter les comportements inattendus pouvant apparaître lors de l'utilisation réelle d'un logiciel.

Ces différentes difficultés ont été surmontées grâce aux échanges réguliers avec mon tuteur, à la documentation disponible et aux recherches personnelles effectuées tout au long du stage.

3.5 Apports du stage

Cette période en entreprise a constitué une expérience particulièrement enrichissante.

Elle m'a permis de découvrir concrètement le fonctionnement d'un service informatique intégré à un Bureau d'Études industriel et de comprendre la manière dont les outils numériques sont utilisés pour répondre aux besoins opérationnels de l'entreprise.

J'ai pu participer à des projets réels destinés à être utilisés quotidiennement par les collaborateurs de SECTRONIC. Cette dimension concrète a été particulièrement motivante et m'a permis de mesurer l'importance de la qualité des développements réalisés.

Ce stage m'a également permis de mieux comprendre l'ensemble du cycle de vie d'une application informatique :

- analyse des besoins ;
- conception de la solution ;
- développement ;
- réalisation des tests ;
- documentation ;
- préparation des évolutions futures.

Au travers des différentes missions réalisées, j'ai pu constater l'importance de la communication entre les utilisateurs et les développeurs afin de concevoir des applications réellement adaptées aux besoins de l'entreprise.

3.6 Conclusion générale

Ce stage effectué au sein de l'entreprise SECTRONIC a constitué une expérience particulièrement formatrice tant sur le plan technique que professionnel.

Au cours de ce stage , j'ai eu l'opportunité de participer à plusieurs projets concrets répondant à des besoins réels de l'entreprise. J'ai notamment contribué au développement d'une application de gestion des visiteurs destinée à remplacer les registres papier, à la réalisation d'une application complète de gestion des congés intégrée à l'intranet ainsi qu'à la validation fonctionnelle du banc de test M-EXTEND.

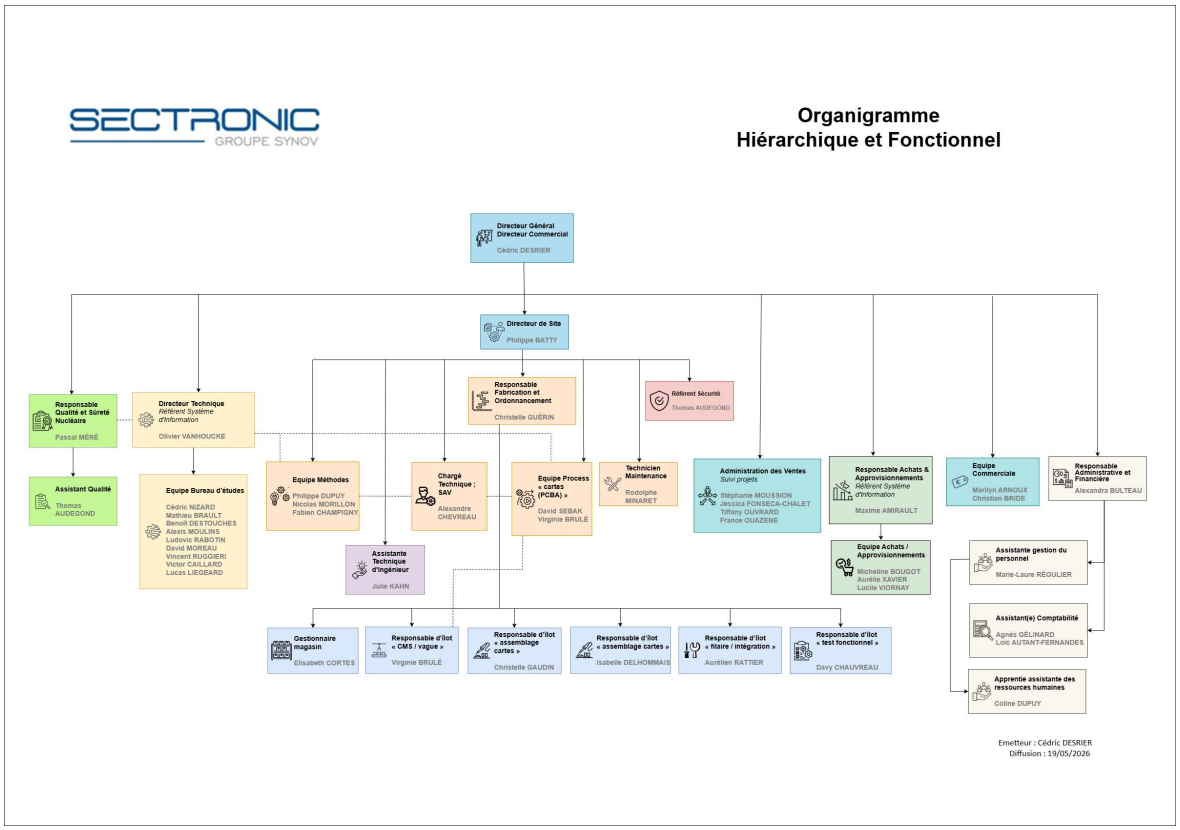
Ces différentes missions m'ont permis d'approfondir mes compétences en développement d'application web avec Django, en conception de bases de données, en intégration de services externes, en automatisation de processus métier et en réalisation de tests logiciels.

Au-delà des aspects techniques, ce stage m'a permis de développer mon autonomie, ma rigueur et ma capacité à travailler dans un environnement professionnel. J'ai également découvert les contraintes liées à la gestion de projets informatiques en entreprise et l'importance de la qualité des solutions développées.

Cette expérience a confirmé mon intérêt pour le développement logiciel et m'a apporté une meilleure compréhension des métiers de l'informatique.

ANNEXES

A.1 Organigramme de l'entreprise :



B. Annexes de la mission 1 :

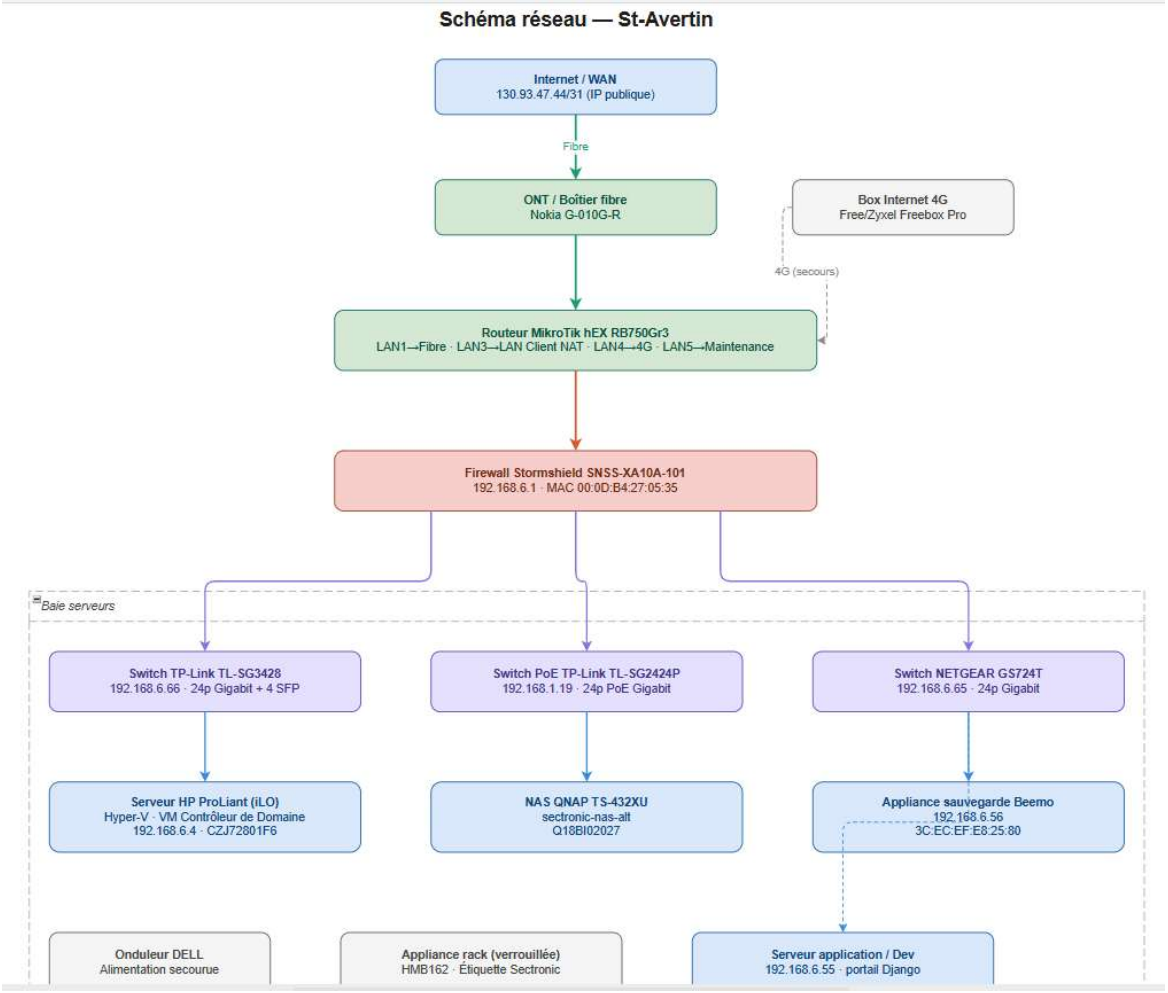
B.1 Extrait tableur inventaire Saint-Avertin

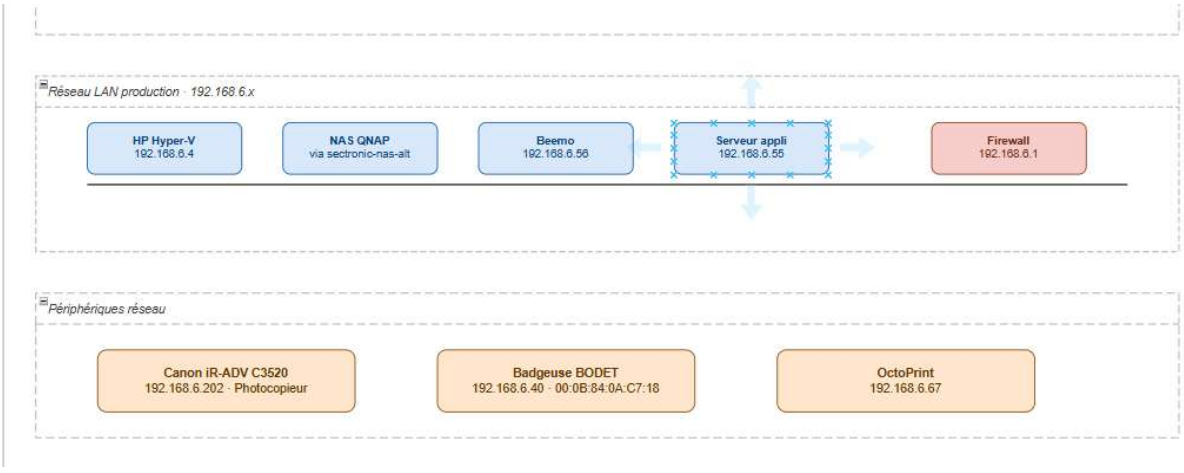
INVENTAIRE MATÉRIEL INFORMATIQUE — St-Avertin

Validé/Pas d'information

N°	Description / Désignation	Marque	Modèle / Référence	Numéro de série	Étiquette HMB	Adresse IP	Adresse MAC	Nom réseau / DNS	Localisation	OS / Licence / Info	État	Date immo	
1	Serveur rack HP (iLO) (Hyperviseur)	HP	ProLiant (DL380 à confirmer)	CZJ72601F6	HMB129			ILOCZJ72601F6	Baie serveurs	Windows Server 2016 STD 16 Cœurs Serveur physique hébergeant l'infrastructure Hyper-V 2016. Contient la VM Contrôleur de Domaine (192.168.6.4)(00:15:5D:03:69:23)	En service		Licence OEM visible ; 843556-425 ; disques sur sticker
2	Appliance rack (tiroir verrouillé)				HMB162				Baie serveurs	Étiquette Sedoo	En service	15/09/22	Équipement verrouillé ; étiquette constructeur
3	NAS rack	QNAP	TS-432XU	Q18BI02027	HMB137		MAC1: 24:5E:BE:36:30:96 MAC2: 24:5E:BE:36:30:97 MAC3: 24:5E:BE:36:30:98 MAC4: 24:5E:BE:36:30:99	sectronic-nas-alt	Baie serveurs	Cloud Key : Q5409-7761	En service		QNAP Systems Inc. ;
4	Firewall / Security Appliance	Stormshield	SNSS-XA10A-101	SNSS1A13B0374A7		192.168.6.1	00:0D:B4:27:05:35		Baie serveurs	Stormshield Network Security (SNS) P/N: SNSS-XA10A-101 Reg. prod: voir étiquette	En service		Restriction accès/cd
5	Routeur	MikroTik	hEX RB750Gr3	HE208XSBC2M/302/r4		IP LAN client : 130.93.47.45/31 IP publique : 130.93.47.44/31	E01 : 48:A9:8A:42:22:98 E05 : 48:A9:8A:42:22:9C		Baie serveurs	Routeur MikroTik LAN1 – Fibre | LAN3 – LAN Client NAT LAN4 – Secours 4G | LAN5 – Maintenance	En service		SECTRONIC STAVER 962
6	ONT / Boîtier fibre	Nokia	G-010G-R	ALCLFIF8AF68			04:B4:BC:2D:7C:94		Baie serveurs	P/N : 3FE49165AAAA S/N secondaire : M082228ALU00057348	En service		
7	Switch gestionable	TP-Link	TL-SG3428			192.168.6.66	10-27-F5-BC-19-37		Baie serveurs	JetStream L2 Managed Switch	En service		24 ports Gigabit + 4 po
8	Switch PoE	TP-Link	TL-SG2424P			192.168.1.19			Baie serveurs	Smart PoE Switch	En service		Switch manageable Pr
9	Switch gestionable	NETGEAR	ProSafe GS724T	5XF1975S00724		192.168.6.65	38:94:ED:B4:F4:2E		Baie serveurs	Smart Switch Gigabit - Firmware v6.3.1.19	En service		24 ports Gigabit
10	Box Internet / passerelle	Free / Zyxel	Freebox Pro — LTE3202-M430	S2022Z0000310			B8:EC:A3:FF:C2:1F		Baie serveurs	Accès Internet professionnel	En service		IMEI : 357407090343 SSID : SectronicOpen

B.2 Schéma réseau Saint-Avertin

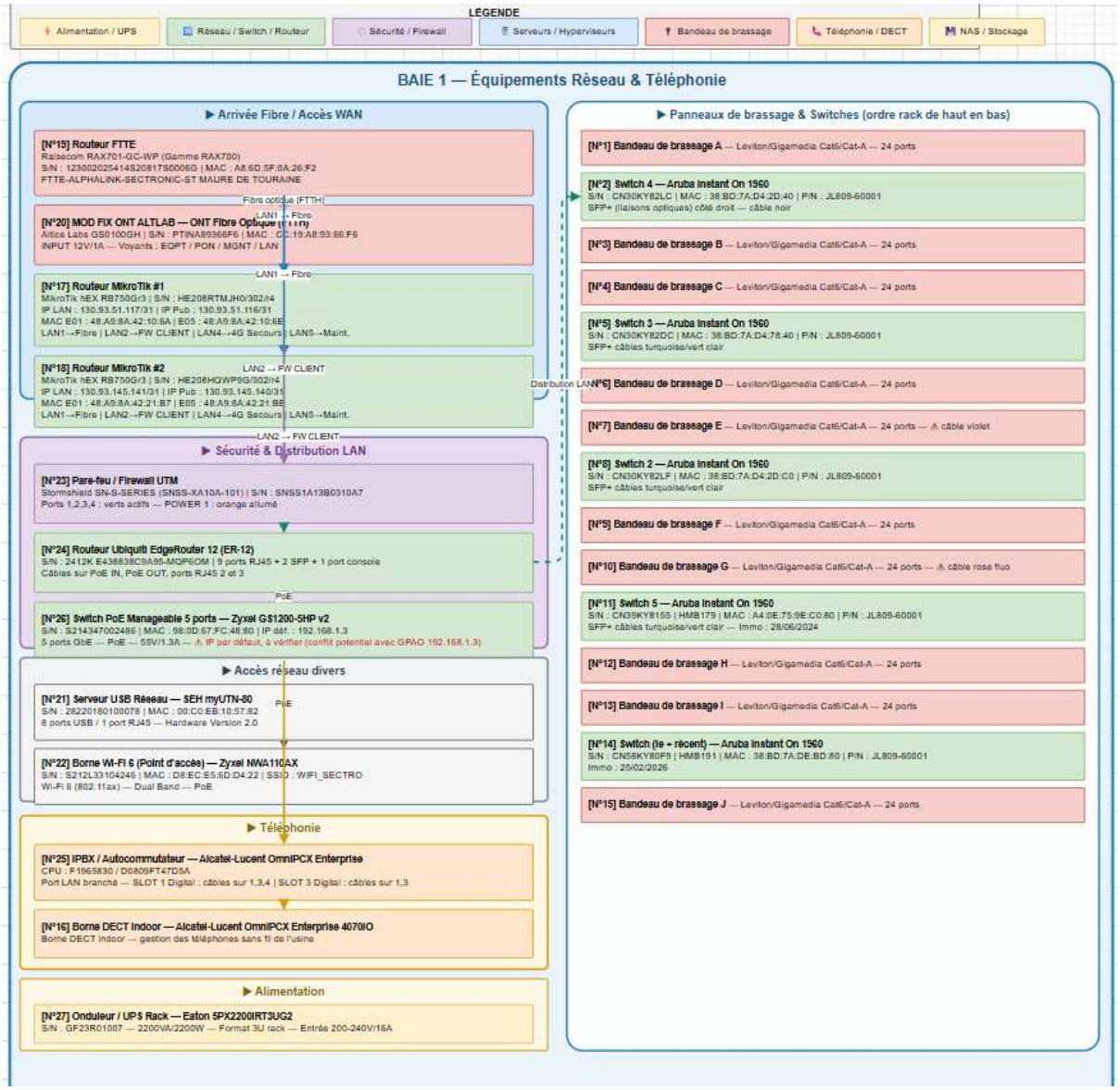


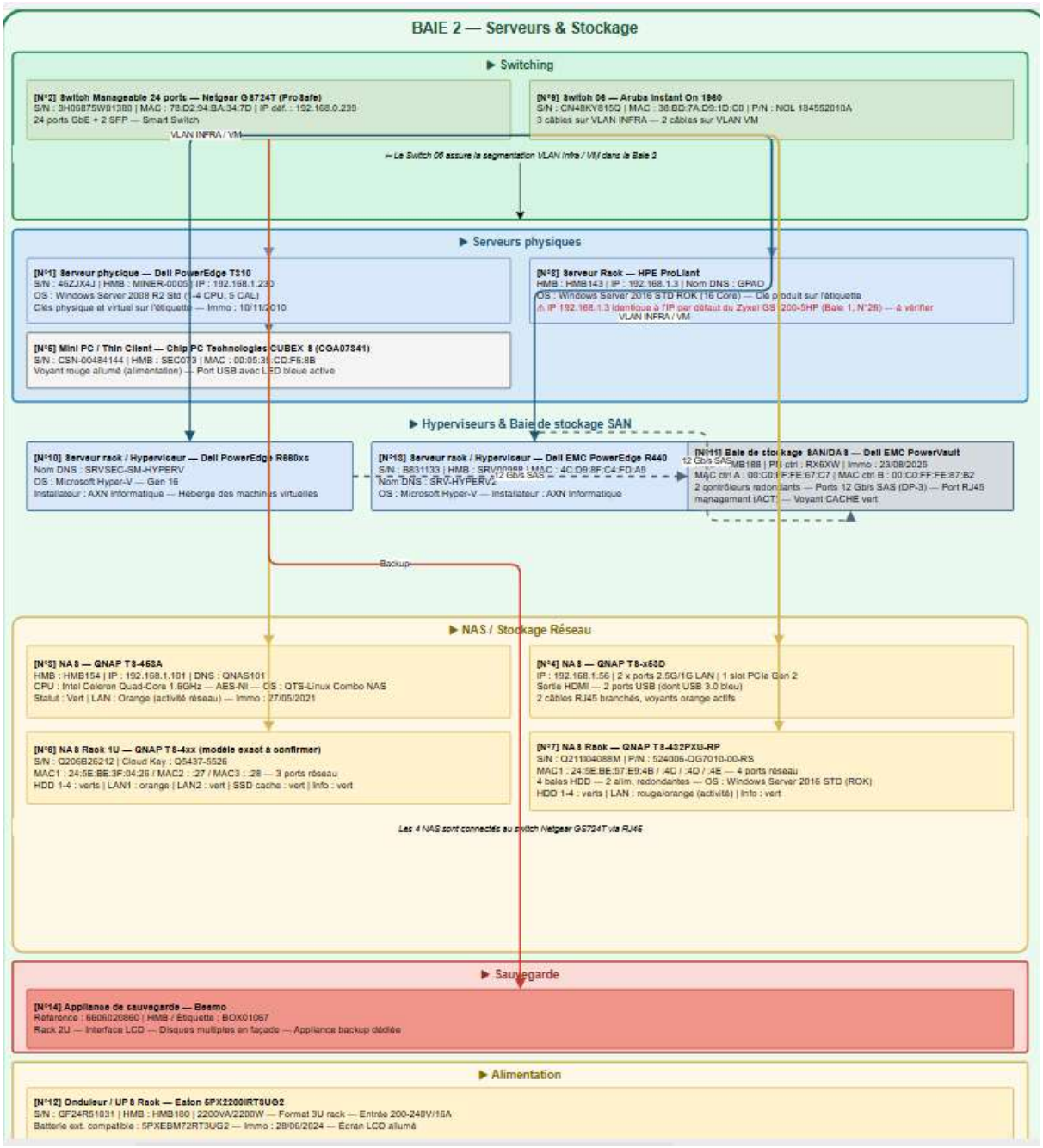


B.3 Extrait tableur inventaire Saint-Maure

2	Switch manageable 24 ports	Netgear	GS724T (ProSafe)	3H06875W01380		Adresse IP par défaut :192.168.0.239	78:D2:94:BA:34:7D		24 ports GbE + 2 SFP — Smart Switch	En service		
3	NAS / Serveur de stockage réseau	QNAP	TS-453A		HMB154	192.168.1.101		QNAS101	CPU : Intel Celeron Quad-Core 1.6GHz Chiffrement : AES-NI accéléré matériellement OS : QTS, Linux Combo NAS	En service	27/05/2021	Statut:Vert. Lan:Orange(activité réseau)
4	NAS / Serveur de stockage réseau	QNAP	TS-x53D			192.168.1.56			Réseau : 2 x ports 2.5G/1G LAN Extension : 1 x slot PCIe Gen 2 Sortie : HDMI	En service		Port HDMI 2 ports USB (dont USB 3.0 bleu) 2 ports RJ45 (2.5G) — 2 câbles branchés, voyants orange actifs Bouton Reset
5	Mini PC / Thin Client	Chip PC Technologies	CUBEX S (CGA07341)	CSN-00484144	SEC073		00:05:35:CD:F6:8B			En service		Voyant rouge allumé (alimentation) Port USB avec LED blanche active
6	NAS rack / Serveur de stockage réseau	QNAP	TS-4xx 1U rack (modèle exact à confirmer)	Q206B26212			24:5E:BE:3F:04:26 (MAC1) / :27 (MAC2) / :28 (MAC3)		4 baies HDD — Cloud Key : Q5437-5526 — 3 ports réseau	En service		HDD 1, 2, 3, 4verts (tous les disques OK)/ LAN 1orange (activité réseau) / LAN 2 vert (connecté) / SSD cache vert / Info vert
7	NAS rack / Serveur de stockage réseau	QNAP	TS-432PXU-RP	Q211I04088M			24:5E:BE:57:E9:4B (mac1) / :4C (MAC2) / :4D (MAC3) / :4E(MAC4)		4 baies — 2 alim. redondantes — 4 ports réseau — Windows Server 2016 STD (ROK)	En service		P/N: 524006-QG7010-00-RS / HDD 1, 2, 3, 4 : verts / LAN : rouge/orange (activité) / Info : vert

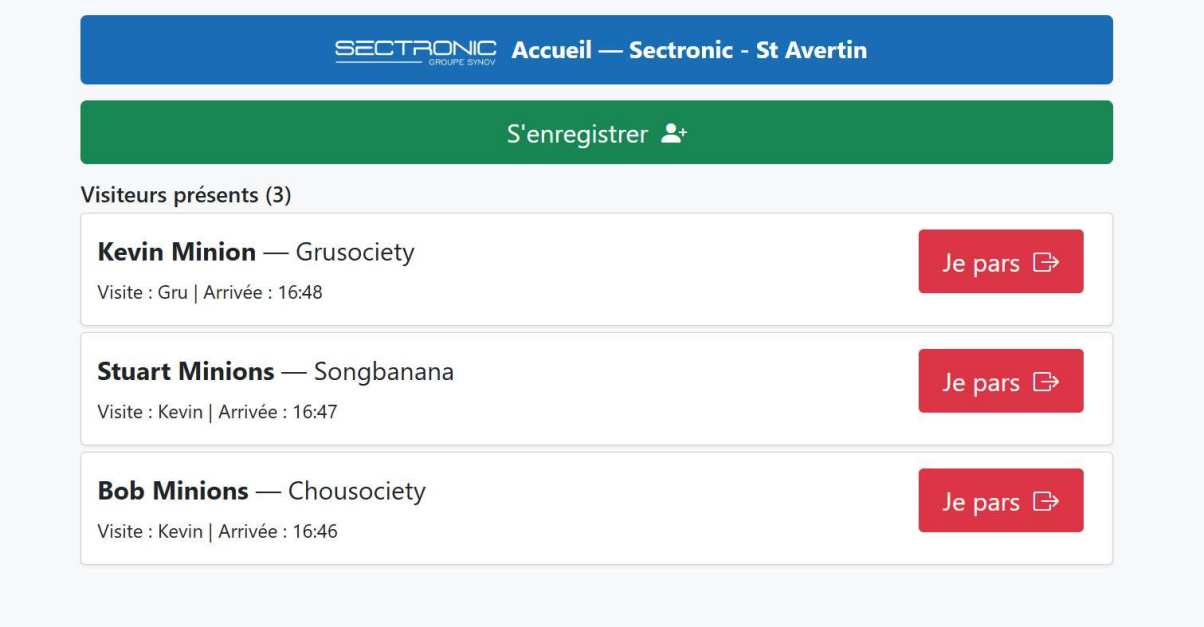
B.4 Schéma réseau Saint-Maure




BAIE 2 — Serveurs & Stockage
▶ Switching
[N°2] Switch Manageable 24 ports — Netgear GS724T (Pro Safe)
S/N : 3H08875W01380 | MAC : 78:D2:94:BA:34:7D | IP déf. : 192.168.0.239
24 ports GbE + 2 SFP — Smart Switch
[N°8] Switch 08 — Aruba Instant On 1960
S/N : CN48KY815Q | MAC : 38:BD:7A:D9:1D:C0 | P/N : NOL 184552010A
3 câbles sur VLAN INFRA — 2 câbles sur VLAN VM
VLAN INFRA / VM
← Le Switch 08 assure la segmentation VLAN Infra / VM dans la Baie 2
▶ Serveurs physiques
[N°1] Serveur physique — Dell PowerEdge T310
S/N : 4BZJX4J | HMB : MINER-0005 | IP : 192.168.1.230
OS : Windows Server 2008 R2 Std (1-4 CPU, 5 CAL)
Clés physique et virtuel sur l'étiquette — Immo : 10/11/2010
[N°5] Serveur Rack — HPE ProLiant
HMB : HMB143 | IP : 192.168.1.3 | Nom DNS : GPAO
OS : Windows Server 2016 STD ROK (16 Core) — Clé produit sur l'étiquette
⚠ IP 192.168.1.3 identique à l'IP par défaut du Zyxel GS 200-5HP (Baie 1, N°25) — à vérifier
VLAN INFRA / VM
[N°6] Mini PC / Thin Client — Chip PC Technologies CUBEX 8 (CGA07S41)
S/N : CSN-00484144 | HMB : SEC073 | MAC : 00:05:3[illegible]:CD:F6:8B
Voyant rouge allumé (alimentation) — Port USB avec LED bleue active
▶ Hyperviseurs & Baie de stockage SAN
[N°10] Serveur rack / Hyperviseur — Dell PowerEdge R660xs
Nom DNS : SRVSEC-SM-HYPERV
OS : Microsoft Hyper-V — Gen 16
Installateur : AXN Informatique — Héberge des machines virtuelles
[N°13] Serveur rack / Hyperviseur — Dell EMC PowerEdge R440
S/N : B831133 | HMB : SRV[illegible] | MAC : 4C:D9:8F:C4:FD:A9
Nom DNS : SRV-HYPERV1
OS : Microsoft Hyper-V — Installateur : AXN Informatique
[N°11] Baie de stockage SAN/DAS — Dell EMC PowerVault
12 Gb/s SAS — MB188 | PN ctrl : RX6XW | Immo : 23/08/2025
MAC ctrl A : 00:C0:FF:FE:67:C7 | MAC ctrl B : 00:C0:FF:FE:87:B2
2 contrôleurs redondants — Ports 12 Gb/s SAS (DP-3) — Port RJ45 management (ACT) — Voyant CACHE vert
12 Gb/s SAS
Backup
▶ NAS / Stockage Réseau
[N°3] NAS — QNAP TS-463A
HMB : HMB154 | IP : 192.168.1.101 | DNS : QNAS101
CPU : Intel Celeron Quad-Core 1.6GHz — AES-NI — OS : QTS-Linux Combo NAS
Statut : Vert | LAN : Orange (activité réseau) — Immo : 27/05/2021
[N°4] NAS — QNAP TS-x53D
IP : 192.168.1.56 | 2 x ports 2.5G/1G LAN | 1 slot PCIe Gen 2
Sortie HDMI — 2 ports USB (dont USB 3.0 bleu)
2 câbles RJ45 branchés, voyants orange actifs
[N°8] NAS Rack 1U — QNAP TS-4xx (modèle exact à confirmer)
S/N : Q206B26212 | Cloud Key : Q5437-5526
MAC1 : 24:5E:BE:3F:04:26 / MAC2 : :27 / MAC3 : :28 — 3 ports réseau
HDD 1-4 : verts | LAN1 : orange | LAN2 : vert | SSD cache : vert | Info : vert
[N°7] NAS Rack — QNAP TS-432PXU-RP
S/N : Q211104088M | P/N : 524006-OG7010-00-RS
MAC1 : 24:5E:BE:57:E9:4B / :4C / :4D / :4E — 4 ports réseau
4 baies HDD — 2 alim. redondantes — OS : Windows Server 2016 STD (ROK)
HDD 1-4 : verts | LAN : rouge/orange (activité) | Info : vert
Les 4 NAS sont connectés au switch Netgear GS724T via RJ45
▶ Sauvegarde
[N°14] Appliance de sauvegarde — Beamo
Référence : 6606020860 | HMB / Étiquette : BOX01067
Rack 2U — Interface LCD — Disques multiples en façade — Appliance backup dédiée
▶ Alimentation
[N°12] Onduleur / UPS Rack — Eaton 5PX2200IRT3UG2
S/N : GF24R51031 | HMB : HMB180 | 2200VA/2200W — Format 3U rack — Entrée 200-240V/16A
Batterie ext. compatible : 5PXEBM72RT3UG2 — Immo : 28/06/2024 — Écran LCD allumé

C. Annexes de la mission 2 :

C.1 Page d'accueil des sites tablettes



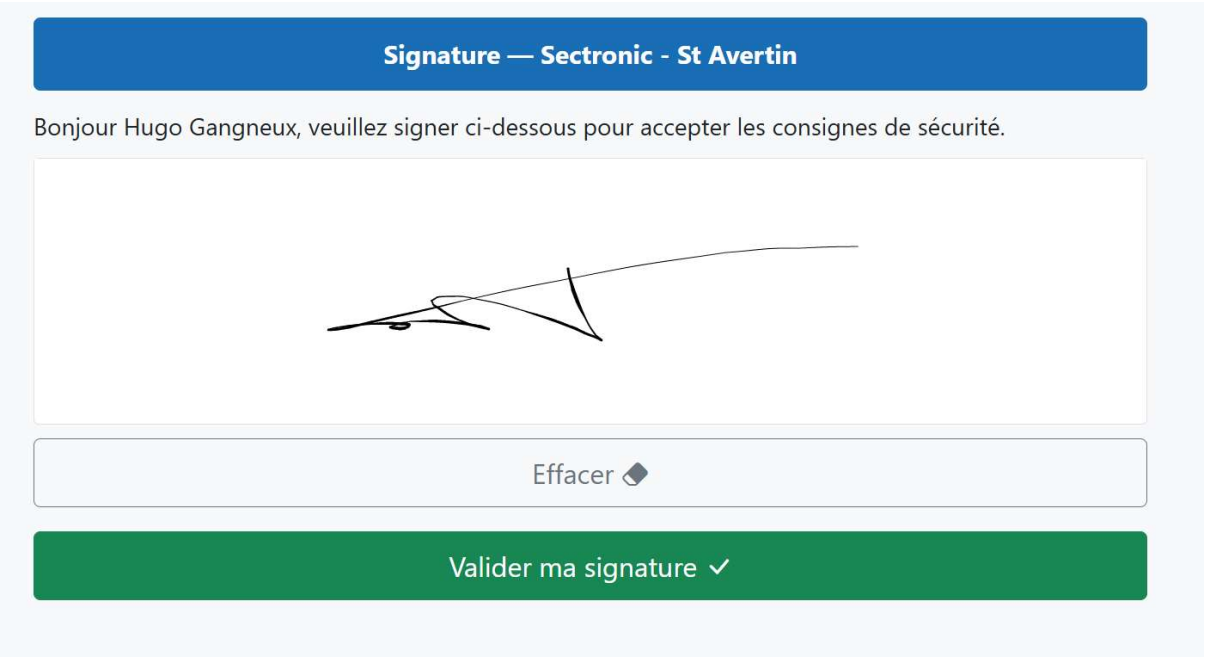
C.2 Formulaire d'enregistrement



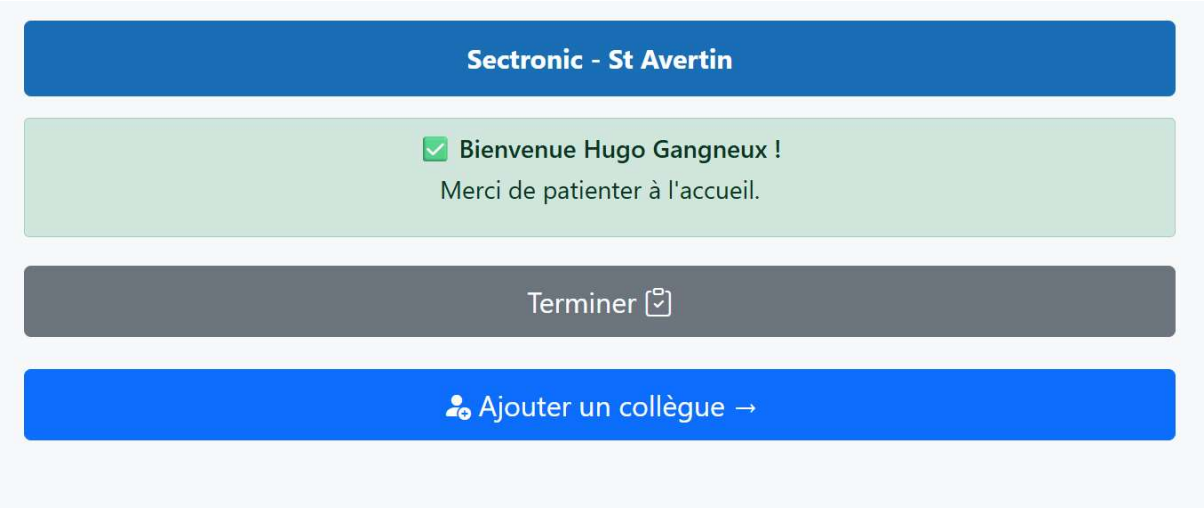
C.3 Consultation des consignes de sécurités



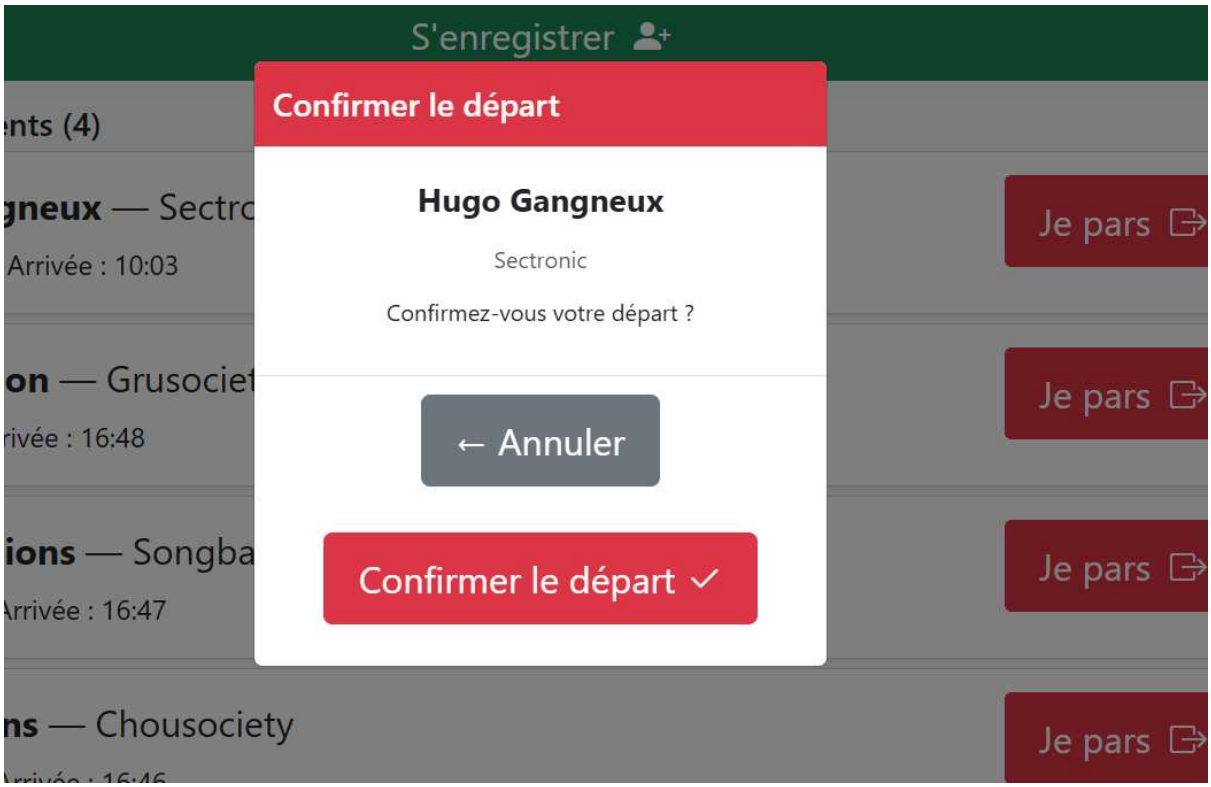
C.4 Signature



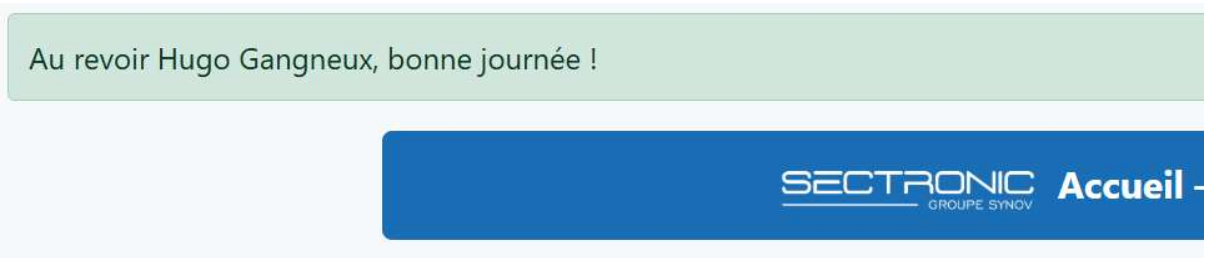
C.5 Page confirmation et ajout collègue



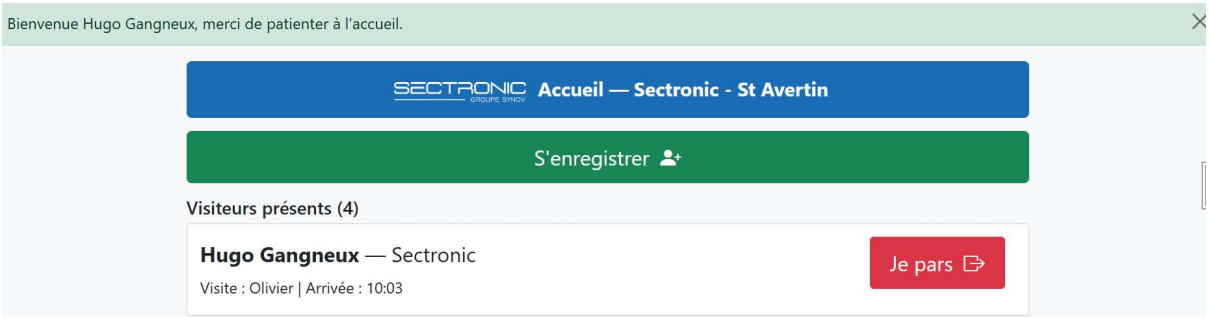
C.6 Confirmation départ



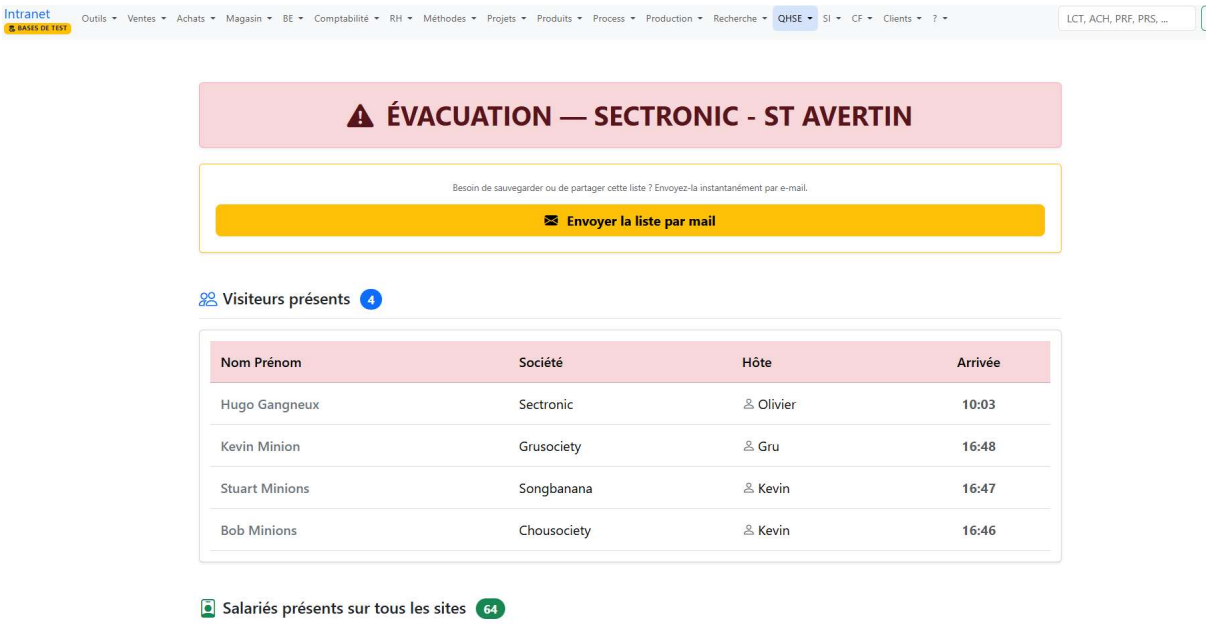
C.7 Message de confirmation de départ



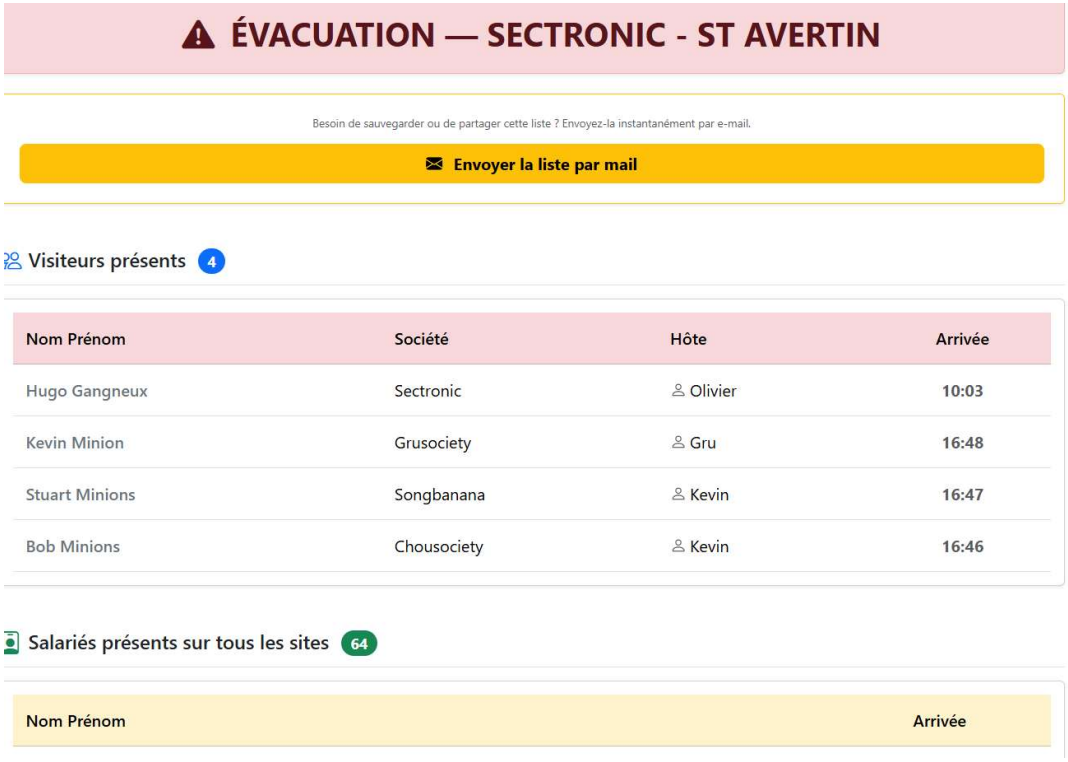
C.8 Confirmation enregistrement message succès



C.9 Page d'évacuation qui est dans l'intranet



C.10 Évacuation salariés



⚠ ÉVACUATION — SECTRONIC - ST AVERTIN

Besoin de sauvegarder ou de partager cette liste ? Envoyez-la instantanément par e-mail.

Envoyer la liste par mail

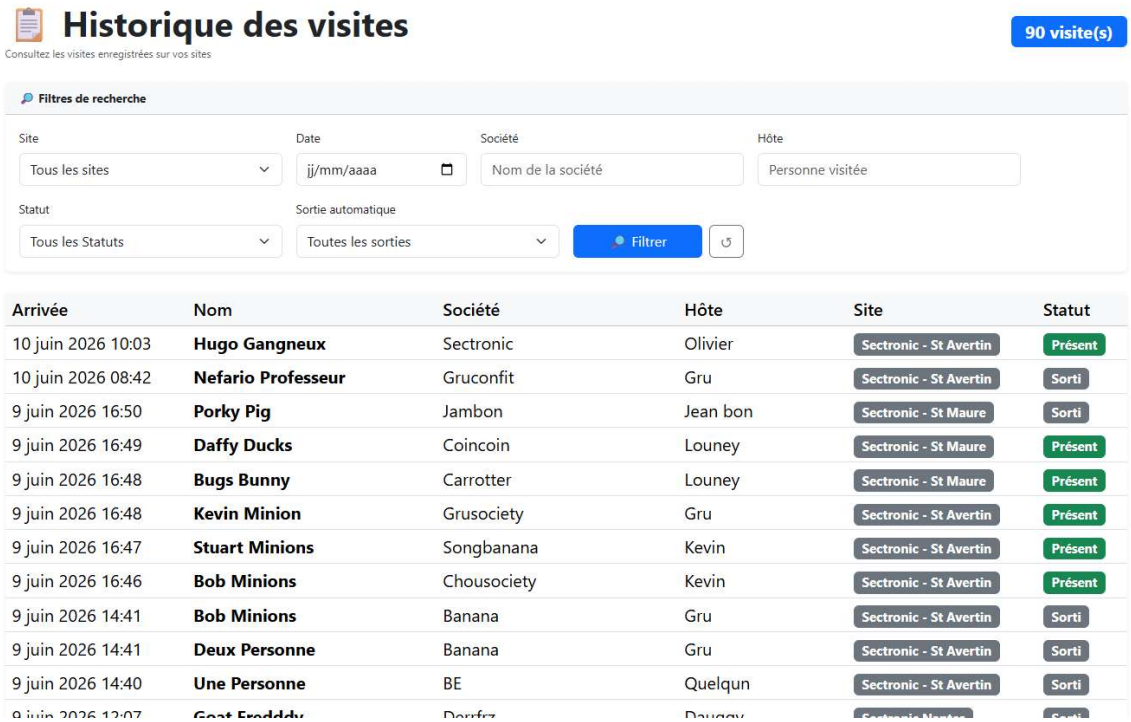
Visiteurs présents 4

Nom Prénom	Société	Hôte	Arrivée
Hugo Gangneux	Sectronic	Olivier	10:03
Kevin Minion	Grusociety	Gru	16:48
Stuart Minions	Songbanana	Kevin	16:47
Bob Minions	Chousociety	Kevin	16:46

Salariés présents sur tous les sites 64

Nom Prénom	Arrivée

C.11 Historique présent dans l'intranet



Historique des visites

Consultez les visites enregistrées sur vos sites

90 visite(s)

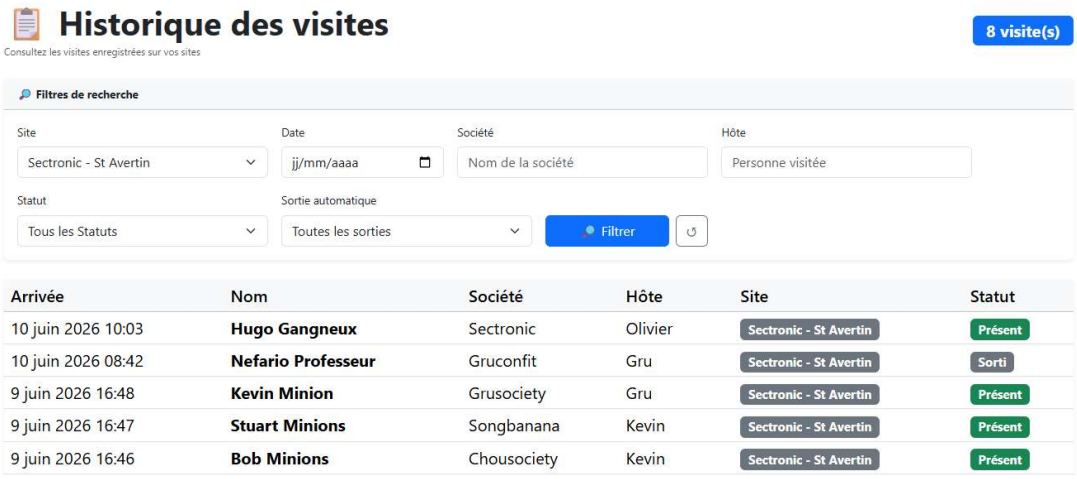
Filtres de recherche

Site: Tous les sites | Date: jj/mm/aaaa | Société: Nom de la société | Hôte: Personne visitée

Statut: Tous les Statuts | Sortie automatique: Toutes les sorties | Filtrer

Arrivée	Nom	Société	Hôte	Site	Statut
10 juin 2026 10:03	**Hugo Gangneux**	Sectronic	Olivier	Sectronic - St Avertin	Présent
10 juin 2026 08:42	**Nefario Professeur**	Gruconfit	Gru	Sectronic - St Avertin	Sorti
9 juin 2026 16:50	**Porky Pig**	Jambon	Jean bon	Sectronic - St Maure	Sorti
9 juin 2026 16:49	**Daffy Ducks**	Coincoin	Louney	Sectronic - St Maure	Présent
9 juin 2026 16:48	**Bugs Bunny**	Carrotter	Louney	Sectronic - St Maure	Présent
9 juin 2026 16:48	**Kevin Minion**	Grusociety	Gru	Sectronic - St Avertin	Présent
9 juin 2026 16:47	**Stuart Minions**	Songbanana	Kevin	Sectronic - St Avertin	Présent
9 juin 2026 16:46	**Bob Minions**	Chousociety	Kevin	Sectronic - St Avertin	Présent
9 juin 2026 14:41	**Bob Minions**	Banana	Gru	Sectronic - St Avertin	Sorti
9 juin 2026 14:41	**Deux Personne**	Banana	Gru	Sectronic - St Avertin	Sorti
9 juin 2026 14:40	**Une Personne**	BE	Quelqun	Sectronic - St Avertin	Sorti

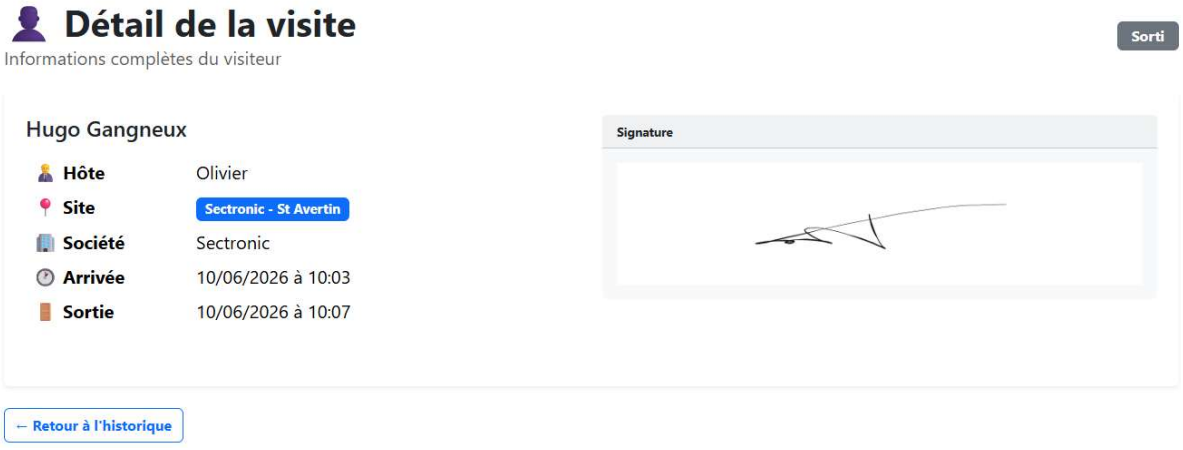
C.12 Filtres de l'historique



C.13 Détail d'un enregistrement dans l'historique



C.14 Détail d'une personne sortie



C.15 Documentation de l'Application Visiteur

Application de gestion des visiteurs pour les sites Sectronic.

L'application remplace le registre papier d'accueil et permet l'enregistrement des visiteurs, la signature électronique des consignes de sécurité, le suivi des présences en temps réel ainsi que la génération de listes d'évacuation.

Objectifs

- Remplacer le registre papier des visiteurs
- Permettre un enregistrement autonome sur tablette tactile
- Faire signer électroniquement les consignes de sécurité
- Suivre les visiteurs présents sur chaque site
- Gérer les départs des visiteurs
- Générer une liste d'évacuation en cas d'urgence
- Envoyer la liste d'évacuation aux responsables du site
- Isoler les visiteurs par site

Architecture

L'application est développée avec Django et intégrée au portail intranet Sectronic.

Technologies utilisées :

- Python 3.12
- Django
- Bootstrap 5
- signature_pad.js pour la signature électronique
- qrcode pour la génération des QR codes
- API Kelio pour la récupération des salariés présents

Stockage

Actuellement :

- SQLite en environnement de développement
- Migration SQL Server prévue

Modèles principaux :

Site

Représente un site Sectronic.

Champ	Type	Description
code	SlugField	Identifiant URL unique (ex : montaigu). Utilisé dans l'URL
nom	CharField	Nom affiché (ex : Sectronic Montaigu)
consignes_url	URLField	URL du PDF des consignes hébergé sur un service externe
emails_evacuation	TextField	Emails séparés par virgules (obligatoire)
responsable_nom	CharField	Nom du responsable
responsable_email	EmailField	Email du responsable

Visiteur

Représente une visite.

Champ	Type	Description
site	ForeignKey	Site d'accueil (PROTECT — suppression interdite si visiteurs liés)
nom	CharField	Nom du visiteur
prenom	CharField	Prénom du visiteur
societe	CharField	Société du visiteur
hote	CharField	Personne visitée
heure_arrivee	DateTimeField	Rempli automatiquement à la création
heure_sortie	DateTimeField	Null si encore présent
sortie_auto	BooleanField	True si marqué sorti par le job automatique
signature	TextField	Signature en base64

Fonctionnement

Arrivée d'un visiteur :

1. Saisie des informations

- nom
- prénom
- société
- personne visitée

2. Consultation des consignes de sécurité

3. Affichage d'un QR Code permettant de récupérer les consignes

4. Signature électronique

5. Validation de la visite

6. Apparition dans la liste des personnes présentes

Arrivée de plusieurs visiteurs

Le mode groupe permet d'ajouter rapidement plusieurs visiteurs de la même société :

- conservation de la société
- conservation de l'hôte
- signature individuelle obligatoire

Départ d'un visiteur

Le visiteur sélectionne :

- « Je pars »

L'heure de sortie est enregistrée et le visiteur disparaît de la liste des présents.

Évacuation

La fonctionnalité d'évacuation est accessible depuis l'intranet Sectronic.

La page évacuation affiche :

- les visiteurs présents sur le site ;
- les salariés présents remontés par Kelio ;
- les informations nécessaires au contrôle des présences.

Un email peut être envoyé aux responsables du site avec la liste complète des personnes présentes.

Multi-sites

Chaque tablette est associée à un site unique.

Exemple :

/visiteur/montaigu/

Les visiteurs sont isolés par site.

Un visiteur enregistré à Nantes n'apparaît jamais sur Montaigu.

Routes principales

Les routes ci-dessous utilisent :

- `<code_site>` : identifiant du site
- `<id>` : identifiant du visiteur

URL	Description
/visiteur/<code_site>/	Accueil tablette
/visiteur/<code_site>/nouveau/	Création visiteur
/visiteur/<code_site>//consignes/	Consignes
/visiteur/<code_site>//signature/	Signature
/visiteur/<code_site>//confirmation/	Confirmation
/visiteur/<code_site>//sortie/	Départ
/visiteur/<code_site>/evacuation/	Liste d'évacuation
/visiteur/<code_site>/evacuation/envoyer/	Envoi email
/visiteur/historique/	Historique
/visiteur/detail/id/	Détail visite

Administration

Les nouveaux sites sont créés depuis l'administration Django.

Informations requises :

- code
- nom
- emails d'évacuation

Une fois créé, le site devient immédiatement disponible via :

/visiteur/code/

Tests

Exécution :

python manage.py test visiteur

Tests couverts

Classe de test	Ce qu'elle vérifie
SiteTestCase	Création d'un site et de ses champs
VisiteurTestCase	Création d'un visiteur lié à un site
SortieVueTestCase	La vue sortie remplit bien le champ heure_sortie
SortieAutoTestCase	Le job daily marque sortie_auto=True
TestIsolationSiteViaView	Un visiteur de Nantes n'apparaît pas sur Montaigu
EvacuationViewKelioMockTests	Page évacuation avec Kelio simulé
SignatureValidationTestCase	Test de validation de la signature

Job automatique

Fichier :

core/jobs/daily/visiteur_auto_sortie.py

Chaque jour à 23h59 :

- fermeture automatique des visites non clôturées ;
- heure_sortie = 23:59 ;
- sortie_auto = True.

Exécution manuelle :

python manage.py runjobs daily --only visiteur_auto_sortie

Déploiement

Les tablettes Android sont configurées en mode kiosque.

URL à utiliser :

http://<serveur>/visiteur/<code_site>/

Développeur

App Django :

visiteur/

Templates :

visiteur/templates/visiteur/

Fichiers statiques :

visiteur/static/visiteur/

Tests :

python manage.py test visiteur

C.16 TODO de l'application Visiteur (Les modifications futurs à faire)

Migration SQL Server

État actuel :

La migration vers SQL Server reste à finaliser.

Points à vérifier :

- migration des modèles `Site` et `Visiteur` ;
- exécution des migrations sur SQL Server ;
- validation des tests après migration (attention, les tests devront être rejoués intégralement).

Enregistrement avant signature

Problème :

Le visiteur est créé en base avant d'avoir signé les consignes de sécurité.

Si l'utilisateur quitte le processus avant la signature :

- visite créée ;
- aucune signature enregistrée.

Cause :

Le modèle est sauvegardé dès la validation du formulaire d'identification.

Solution envisagée :

Stocker temporairement les informations en session Django, puis créer le visiteur uniquement après validation de la signature.

Priorité Moyenne

Soumissions multiples (double-tap)

Problème :

Un utilisateur peut appuyer plusieurs fois rapidement sur les boutons de validation.

Conséquences :

- doublons de visiteurs ;
- signatures manquantes ;
- accumulation des messages de succès ;
- incohérence des listes de présence.

Cause :

Absence de protection contre les soumissions multiples.

Solutions envisagées :

Front-end :

- désactiver le bouton après le premier clic ;
- afficher un indicateur de chargement.

Back-end :

- mettre en place un mécanisme d'idempotence ;
- empêcher plusieurs créations identiques dans un court intervalle.

Priorité Haute

Bouton « Je pars »

Problème :

Plusieurs clics rapides génèrent plusieurs notifications successives.

Solution envisagée :

Désactivation immédiate du bouton après soumission.

Priorité Faible

Filtrage des salariés par site (Kelio)

Problème :

L'API Kelio utilisée pour récupérer les salariés présents ne permet actuellement pas d'identifier avec certitude le site de présence d'un salarié.

Lors d'une évacuation, la liste des salariés affichée contient donc l'ensemble des salariés présents remontés par Kelio, sans distinction entre les différents sites Sectronic.

Conséquences :

- les listes d'évacuation peuvent contenir des salariés appartenant à d'autres sites ;
- les responsables doivent effectuer un tri manuel lors du contrôle des présences ;
- les informations affichées ne reflètent pas précisément l'occupation réelle du site concerné.

Solution envisagée :

Étudier les possibilités offertes par Kelio pour :

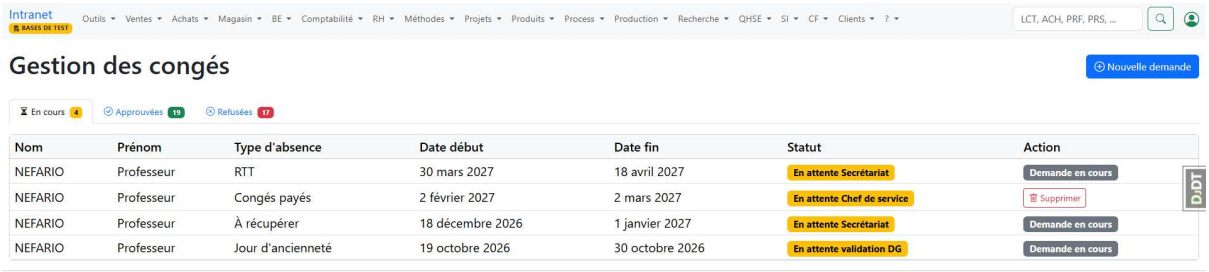
- récupérer le site de présence d'un salarié ;
- exploiter une information de badgeage ou de localisation ;
- mettre en place un mapping entre les données Kelio et les sites de l'application.

Une fois cette information disponible, filtrer les salariés afin de n'afficher que ceux présents sur le site concerné par l'évacuation.

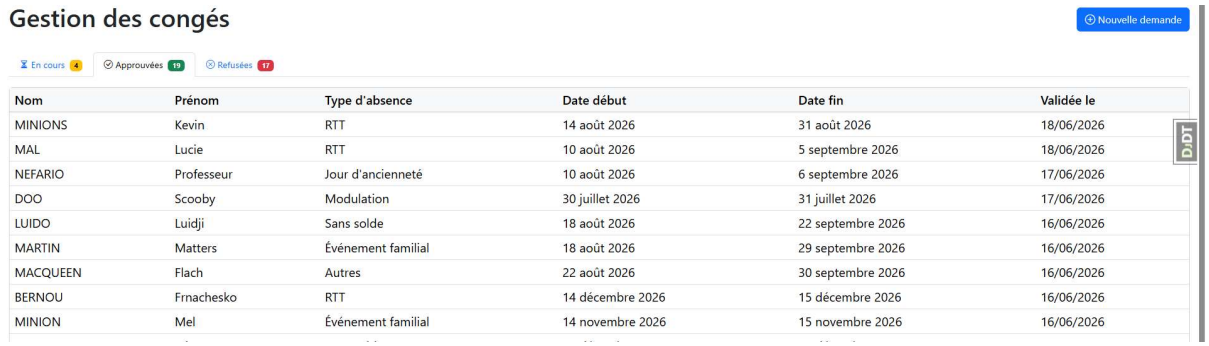
Priorité Moyenne

D. Annexes de la mission 3 :

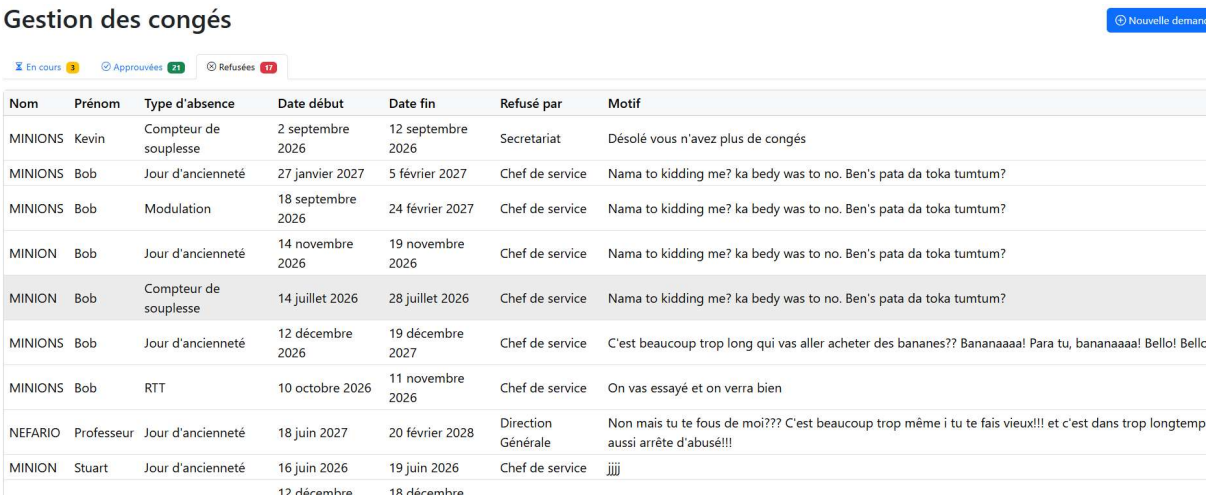
D.1 Accueil demande d'absences



D.2 Accueil onglet approuvée



D.3 Accueil onglet refusé

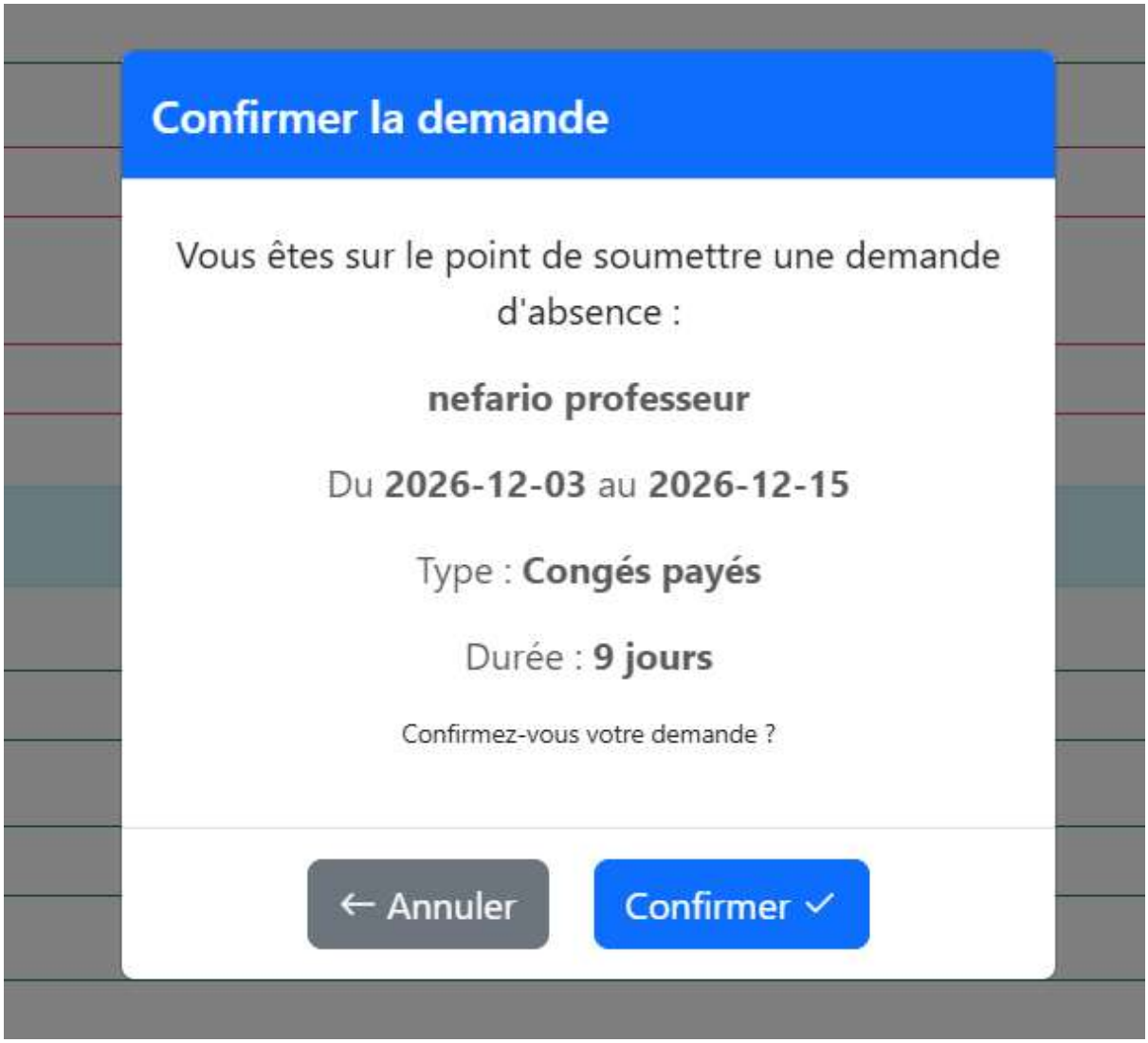


Nom	Prénom	Type d'absence	Date début	Date fin	Refusé par	Motif
MINIONS	Kevin	Compteur de souplesse	2 septembre 2026	12 septembre 2026	Secretariat	Désolé vous n'avez plus de congés
MINIONS	Bob	Jour d'ancienneté	27 janvier 2027	5 février 2027	Chef de service	Nama to kidding me? ka bedy was to no. Ben's pata da toka tumtum?
MINIONS	Bob	Modulation	18 septembre 2026	24 février 2027	Chef de service	Nama to kidding me? ka bedy was to no. Ben's pata da toka tumtum?
MINION	Bob	Jour d'ancienneté	14 novembre 2026	19 novembre 2026	Chef de service	Nama to kidding me? ka bedy was to no. Ben's pata da toka tumtum?
MINION	Bob	Compteur de souplesse	14 juillet 2026	28 juillet 2026	Chef de service	Nama to kidding me? ka bedy was to no. Ben's pata da toka tumtum?
MINIONS	Bob	Jour d'ancienneté	12 décembre 2026	19 décembre 2027	Chef de service	C'est beaucoup trop long qui vas aller acheter des bananes?? Bananaaaa! Para tu, bananaaaa! Bello! Bellc
MINIONS	Bob	RTT	10 octobre 2026	11 novembre 2026	Chef de service	On vas essayé et on verra bien
NEFARIO	Professeur	Jour d'ancienneté	18 juin 2027	20 février 2028	Direction Générale	Non mais tu te fous de moi??? C'est beaucoup trop même i tu te fais vieux!!! et c'est dans trop longtemp aussi arrête d'abusé!!!
MINION	Stuart	Jour d'ancienneté	16 juin 2026	19 juin 2026	Chef de service	jjjj

D.4 Formulaire demande d'absences



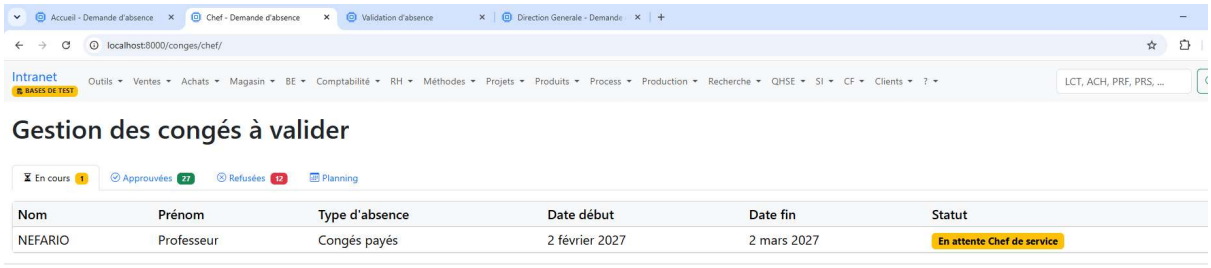
D.5 Confirmation demande d’absences



D.6 Confirmation annulation d’une demande une fois qu’elle à été faites



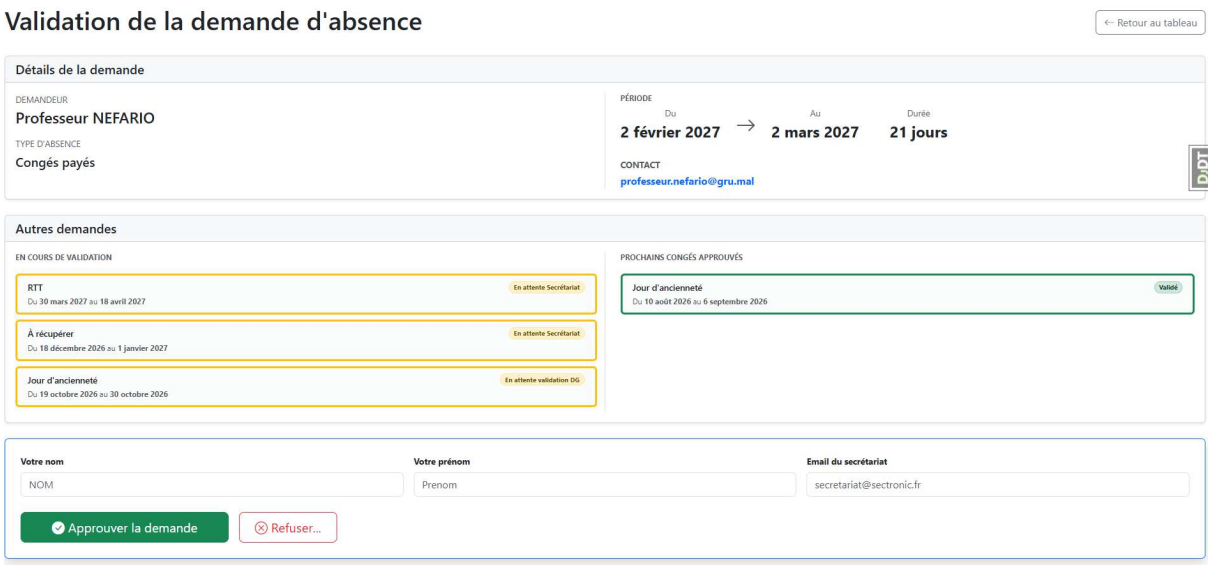
D.7 Accueil Chef, secrétariat, Direction générale



D.8 Chef, secrétariat, Direction générale onglet approuvée



D.9 Validation d'une absences



D.10 Validation d'une absence partie détail



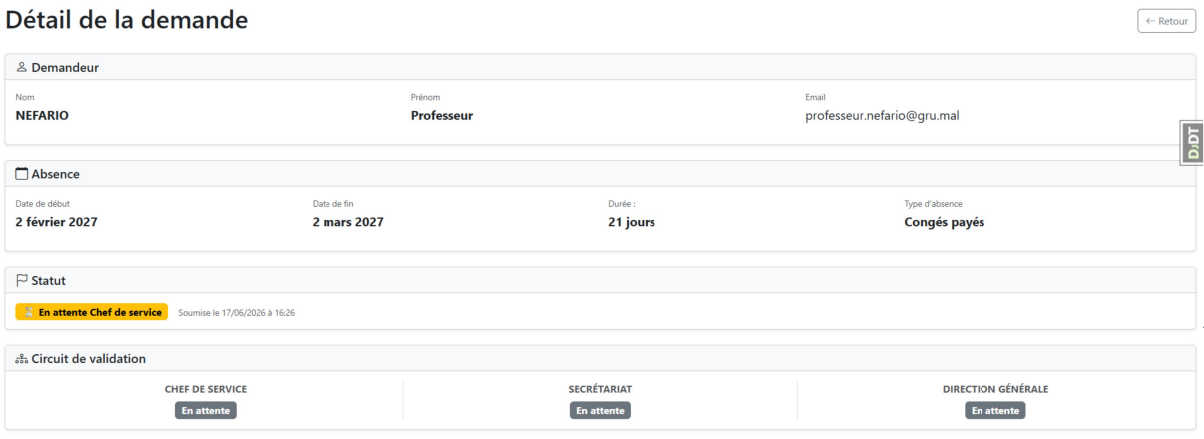
D.11 Validation d'une absence partie autres demandes



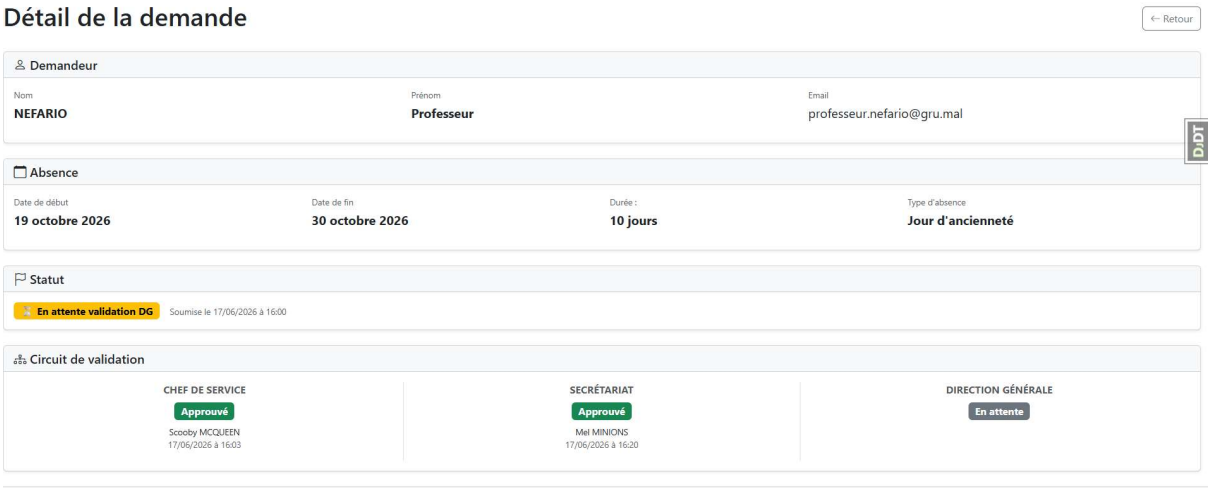
D.12 Validation d'une absence partie choix



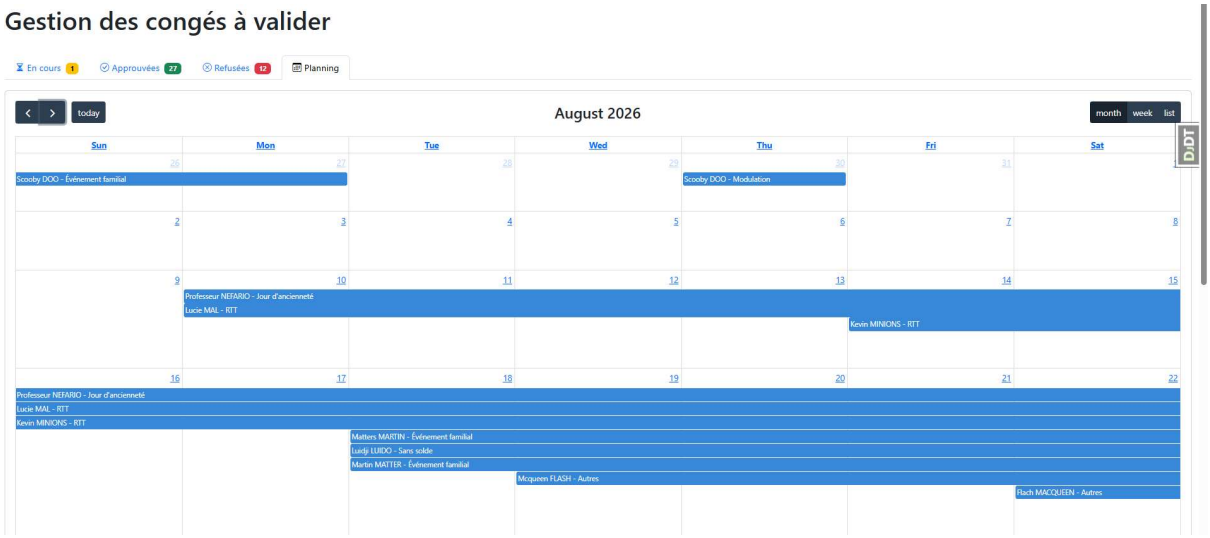
D.13 Détail d'une demande en cours



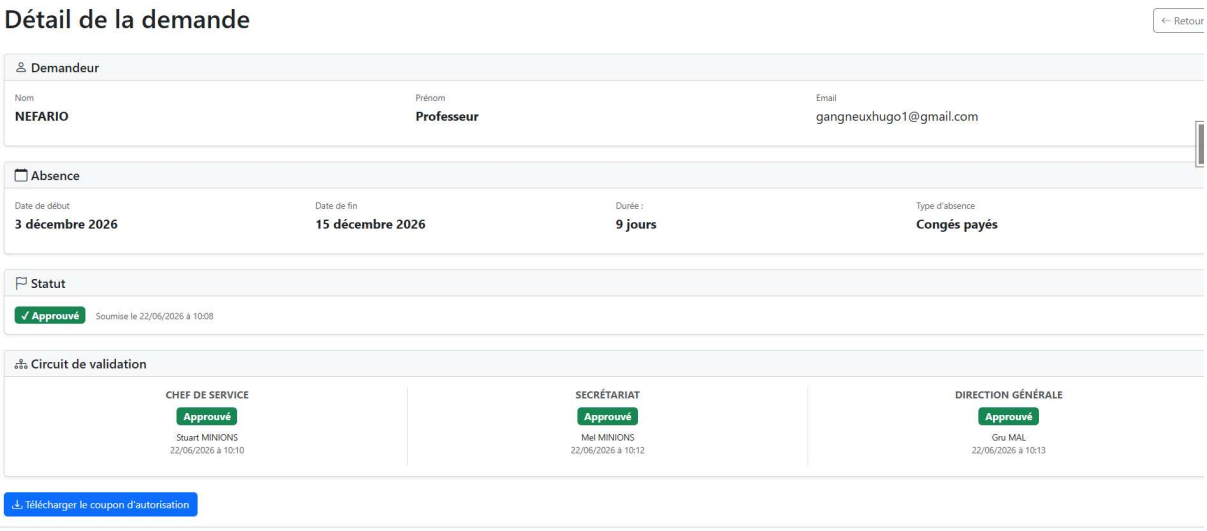
D.14 Détail d'une demande en attente DG



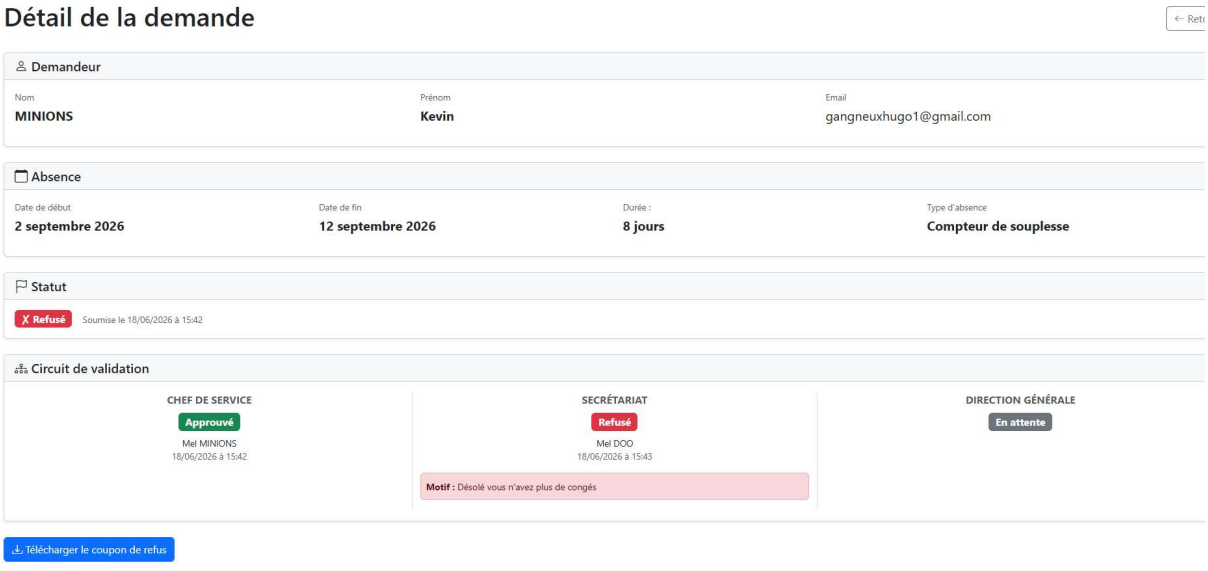
D.15 Calendrier des absences disponible pour le chef, le secrétariat et la DG



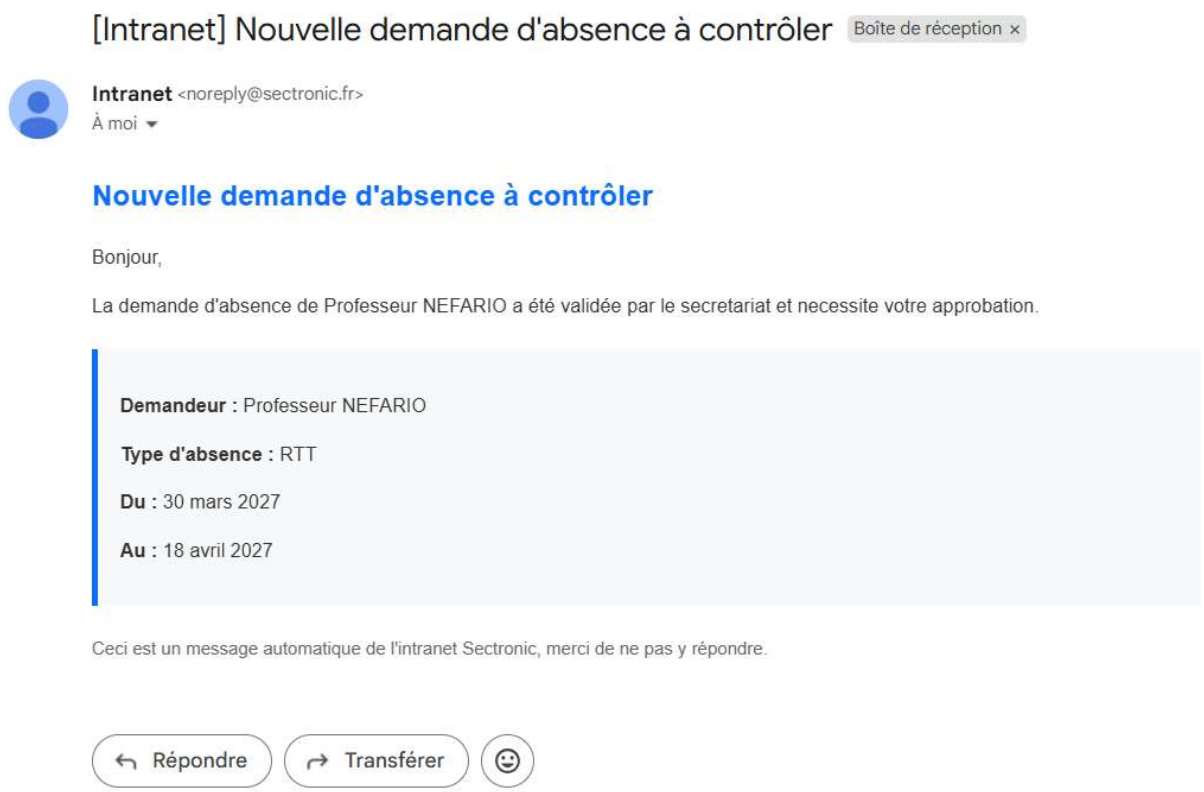
D.16 Détail d'une demande validée



D.17 Détail d'une demande refusée



D.18 Mail demande d’absence à valider



[Intranet] Nouvelle demande d'absence à contrôler Boîte de réception ×

Intranet <noreply@sectronic.fr>
À moi

Nouvelle demande d'absence à contrôler

Bonjour,

La demande d'absence de Professeur NEFARIO a été validée par le secretariat et necessite votre approbation.

Demandeur : Professeur NEFARIO

Type d'absence : RTT

Du : 30 mars 2027

Au : 18 avril 2027

Ceci est un message automatique de l'intranet Sectronic, merci de ne pas y répondre.

Répondre Transférer

D.19 Mail demande d’absence approuvée définitivement

[Intranet] Votre demande d'absence a été approuvée Boîte de réception ×

Intranet <noreply@sectronic.fr>
À moi

Approbation de votre demande d'absence

Bonjour,

Votre demande d'absence du 2026-12-03 au 2026-12-15 a été validée. Vous trouverez ci-joint votre coupon d'autorisation.

Demandeur : Professeur NEFARIO

Type d'absence : Congés payés

Du : 3 décembre 2026

Au : 15 décembre 2026

Ceci est un message automatique de l'intranet Sectronic, merci de ne pas y répondre.

2 pièces jointes • Analysé par Gmail Tout ajouter à Drive

D.20 PDF demande absence validée

DEMANDE D'AUTORISATION D'ABSENCE

NOM : NEFARIO **Prénom :** Professeur

Je soussigné(e) sollicite l'autorisation de m'absenter :
Du : 03/12/2026 **Au :** 15/12/2026

Motif : Congés payés

Avis Chef de service	Contrôle (Secrétariat)	Avis D.G.
Stuart MINIONS 22/06/2026	Mel MINIONS 22/06/2026	Gru MAL 22/06/2026

La demande d'autorisation d'absence a été : ACCEPTÉE

Validée définitivement le 22/06/2026

Document généré automatiquement par l'intranet Sectronic

D.21 PDF demande absence refusé



DEMANDE D'AUTORISATION D'ABSENCE

NOM : MINIONS **Prénom :** Kevin

Absence demandée :
Du : 02/09/2026 **Au :** 12/09/2026

Motif : Compteur de souplesse

Refusée par : Mel DOO (Secrétariat)
Le : 18/06/2026

Motif du refus :

Désolé vous n'avez plus de congés

· DEMANDE REFUSÉE Refusée le 18/06/2026

Document généré automatiquement par l'intranet Sectronic

D.22 Documentation de l'Application Congés

Application de gestion des demandes d'absence intégrée à l'intranet de l'entreprise.

L'application permet aux collaborateurs de soumettre des demandes d'absence et de suivre un processus de validation en plusieurs étapes impliquant :

- le Chef de service ;
- le Secrétariat ;
- la Direction Générale.

Une fois validée, la demande est approuvée définitivement, un document PDF est généré automatiquement et envoyé au demandeur par email.

Objectifs

- Dématérialiser les demandes d'absence ;
- Centraliser le suivi des congés ;
- Assurer la traçabilité des validations ;
- Automatiser les notifications par email ;
- Réduire les échanges manuels ;
- Fournir un historique des demandes ;
- Permettre une validation hiérarchique en plusieurs étapes.

Architecture

L'application est développée avec Django et intégrée au portail intranet.

Technologies utilisées :

- Python 3.x
- Django
- Bootstrap 5
- JavaScript
- FullCalendar
- Génération PDF
- Module interne `core.emailing`

Stockage actuel :

- SQLite en développement

Compatible avec une future migration SQL Server.

Modèle principal

DemandeAbsence

Représente une demande d'absence ainsi que son cycle complet de validation.

Informations du demandeur

Champ	Type	Description
nom	CharField	Nom du demandeur
prenom	CharField	Prénom du demandeur
email	EmailField	Adresse email du demandeur

Période d'absence

Champ	Type	Description
date_debut	DateField	Date de début
date_fin	DateField	Date de fin
heure_debut	TimeField	Heure de début (absence courte)
heure_fin	TimeField	Heure de fin (absence courte)

Types d'absence

Valeur	Libellé
conges_payes	Congés payés
rtt	RTT
anciennete	Jour d'ancienneté
sans_solde	Sans solde
modulation	Modulation

Valeur	Libellé
souplesse	Compteur de souplesse
a_recuperer	À récupérer
familial	Événement familial
autres	Autres

Informations complémentaires

Champ	Type	Description
motif_autres	TextField	Précision lorsque le type est "Autres"
date_recuperation	DateField	Date de récupération associée
justificatif	TextField	Justificatif d'événement familial

Statuts

Valeur	Libellé
en_attente_chef	En attente Chef de service
en_attente_secretariat	En attente Secrétariat
en_attente_dg	En attente validation DG
approuve	Approuvé
refuse	Refusé

Suivi de la demande

Champ	Type
statut	CharField
date_soumission	DateTimeField

Validation Chef de service

Champ
chef_nom
chef_prenom
chef_date
chef_motif_refus
email_chef

Validation Secrétariat

Champ
secretariat_nom
secretariat_prenom
secretariat_date
secretariat_motif_refus
email_secretariat

Validation Direction Générale

Champ
dg_nom
dg_prenom
dg_date
dg_motif_refus
email_dg

Fonctionnement

Soumission d'une demande :

L'employé renseigne :

- nom ;
- prénom ;
- email ;
- période d'absence ;
- type d'absence ;
- informations complémentaires éventuelles.

Après validation :

- création de la demande ;
- statut = En attente Chef de service ;
- envoi automatique d'un email au chef de service.

Validation Chef de service

Le responsable consulte la demande et peut :

- approuver ;
- refuser.

En cas de refus :

- saisie obligatoire d'un motif ;
- statut = Refusé ;
- notification au demandeur.

En cas d'approbation :

- statut = En attente Secrétariat ;
- notification au secrétariat.

Validation Secrétariat

Le secrétariat vérifie la demande.

Actions possibles :

- valider ;
- refuser.

En cas de refus :

- saisie obligatoire d'un motif ;
- statut = Refusé ;
- notification au demandeur avec coupon PDF de refus en pièce jointe.

En cas de validation :

- statut = En attente DG ;
- notification à la Direction Générale.

Validation Direction Générale

La DG effectue la validation finale.

Actions possibles :

- approuver définitivement ;
- refuser.

En cas de refus :

- saisie obligatoire d'un motif ;
- statut = Refusé ;
- notification au demandeur.

En cas d'approbation :

- statut = Approuvé ;
- génération automatique du PDF d'autorisation ;
- envoi du PDF au salarié ;
- archivage de la validation.

Workflow de validation

Employé
  |
  v
En attente Chef de service
  |
  +---- Refus ----> Refusé + PDF refus envoyé
  |

v

En attente Secrétariat

|

+---- Refus ----> Refusé + PDF refus envoyé

|

v

En attente DG

|

+---- Refus ----> Refusé + PDF refus envoyé

|

v

Approuvé

|

+---- Génération PDF autorisation

|

+---- Notification email + PDF en pièce jointe

Notifications

Les notifications sont gérées par le module interne :

core.emailing

Emails envoyés automatiquement :

- nouvelle demande au chef de service ;
- validation chef vers secrétariat ;
- validation secrétariat vers DG ;
- approbation finale vers le salarié avec PDF d'autorisation en pièce jointe ;
- refus vers le salarié avec PDF de refus en pièce jointe.

Génération PDF :

Deux types de coupons PDF sont générés automatiquement.

Coupon d'autorisation (demande approuvée) :

- généré lors de la validation finale par la DG ;
- envoyé automatiquement par email au salarié ;
- téléchargeable depuis la page de détail de la demande.

Coupon de refus (demande refusée) :

- généré lors d'un refus à n'importe quelle étape ;
- adapté automatiquement selon le rôle du validateur (Chef de service, Secrétariat ou DG) ;
- envoyé automatiquement par email au salarié ;

- téléchargeable depuis la page de détail de la demande.

Calcul de la durée d'absence

Un fichier `utils.py` centralise le calcul de la durée d'une absence.

La fonction `calculer_duree_absence` gère deux cas :

- absence sur une seule journée avec horaires renseignés : calcul en heures et minutes ;
- absence sur plusieurs jours : calcul en jours ouvrés (lundi au vendredi, week-ends exclus).

La durée est affichée dans les pages de détail et dans les interfaces de validation.

Sécurisation des formulaires

Afin d'éviter les soumissions multiples accidentelles, une protection a été mise en place côté JavaScript.

Lors du clic sur le bouton de confirmation, le formulaire est soumis et les interactions utilisateur sont immédiatement bloquées via la propriété `pointerEvents`.

Cette solution empêche efficacement les doubles clics et les envois multiples de notifications par email.

Calendrier des absences

L'application utilise FullCalendar afin d'afficher les absences validées sous forme de calendrier interactif accessible depuis les tableaux de bord du Chef de service, du Secrétariat et de la Direction Générale.

Les données sont transmises au JavaScript via une balise :

<script type="application/json">

afin d'éviter les problèmes d'échappement des caractères spéciaux.

Cette fonctionnalité pourra être enrichie lorsque les rôles utilisateurs seront mis en place afin de filtrer automatiquement les collaborateurs visibles selon le responsable connecté.

Routes principales

Les routes ci-dessous utilisent :

- `<pk>` : identifiant d'une demande d'absence

URL	Description
/conges/	Accueil
/conges/soumettre/	Création d'une demande
/conges/annuler//	Annulation d'une demande
/conges/detail//	Détail d'une demande
/conges/chef/	Tableau de bord Chef de service
/conges/chef//	Validation Chef de service
/conges/secretariat/	Tableau de bord Secrétariat
/conges/secretariat//	Validation Secrétariat

URL	Description
/conges/dg/	Tableau de bord Direction Générale
/conges/dg//	Validation Direction Générale

Fonctionnalités actuellement disponibles

Employé :

- création d'une demande avec modal de confirmation ;
- consultation du détail ;
- suivi du statut ;
- annulation d'une demande en cours (statut En attente Chef uniquement) ;
- voir les demandes approuvées ;
- voir les demandes refusées avec motif ;
- téléchargement du coupon PDF (autorisation ou refus).

Chef de service :

- consultation des demandes en attente ;
- validation ou refus avec motif ;
- voir les absences refusées ;
- voir les absences validées et leur état d'avancement ;
- planning de toutes les absences approuvées (FullCalendar) ;
- consultation du détail d'une demande ;
- visibilité sur les demandes en cours et futures de l'employé concerné.

Secrétariat :

- contrôle administratif ;
- validation ou refus avec motif ;
- voir les absences refusées ;
- voir les absences validées et leur état d'avancement ;
- planning de toutes les absences approuvées (FullCalendar) ;
- consultation du détail d'une demande ;
- visibilité sur les autres demandes en cours et futures de l'employé concerné.

Direction Générale :

- validation finale ou refus ;
- voir les absences refusées ;
- voir les absences validées ;
- planning de toutes les absences approuvées (FullCalendar) ;
- consultation du détail d'une demande ;

- visibilité sur les autres demandes en cours et futures de l'employé concerné ;
- génération et envoi du PDF d'autorisation.

Limitations actuelles

L'application fonctionne actuellement sans système d'authentification.

Par conséquent :

- les informations sont saisies manuellement ;
- les rôles ne sont pas vérifiés ;
- les tableaux de bord ne sont pas isolés par utilisateur ;
- les accès aux URLs ne sont pas protégés.

Ces limitations seront corrigées lors de l'intégration du système de connexion de l'intranet.

Fonctionnalités prévues

- authentification intranet ;
- gestion des rôles ;
- restriction des accès ;
- notifications intranet ;
- filtrage du planning par équipe pour le chef de service ;
- statistiques et indicateurs.

Développeur

Application Django :

conges/

Templates :

conges/templates/conges/

Fichiers statiques :

conges/static/conges/

Tests :

python manage.py test conges

D.22 TODO de l'Application Congés

Gestion des rôles et authentification

Problème :

L'application fonctionne actuellement sans système d'authentification.

Toutes les informations sont saisies manuellement par les utilisateurs et aucune restriction d'accès n'est appliquée selon le rôle de la personne connectée.

Conséquences :

Actuellement :

- n'importe quel utilisateur peut accéder aux pages Employé, Chef de service, Secrétariat et DG ;
- aucune page n'est protégée ;
- aucune vérification ne permet de garantir que l'utilisateur possède réellement le rôle affiché ;
- les noms, emails et informations de validation peuvent être saisis librement ;
- les actions de validation ne sont pas associées à une identité vérifiée.

Impact futur :

Lorsque le système de connexion de l'intranet sera disponible, une refonte importante de l'application sera nécessaire :

- modèles Django ;
- routes ;
- vues ;
- formulaires ;
- templates ;
- règles de sécurité.

Chaque utilisateur devra uniquement accéder aux informations correspondant à son rôle.

Priorité Haute

Isolation des données par utilisateur

Problème :

Toutes les demandes sont actuellement visibles par tous les utilisateurs.

Conséquences :

Un employé peut consulter :

- les demandes des autres employés ;
- les statuts ;
- les dates ;
- les informations de validation.

Solution envisagée :

Associer chaque demande à l'utilisateur connecté.

L'employé ne devra voir que :

- ses demandes en cours ;
- son historique ;
- les validations associées.

Priorité Haute

Isolation des validations Chef de service

Problème :

Tous les chefs de service partagent actuellement le même tableau de validation.

Conséquences :

Un chef peut :

- consulter les demandes d'équipes qu'il ne supervise pas ;
- approuver ou refuser des demandes qui ne lui appartiennent pas.

Solution envisagée :

Associer chaque employé à son responsable hiérarchique.

Le tableau de bord chef devra uniquement afficher :

- les demandes de son équipe ;
- son historique de validation ;
- son planning d'équipe.

Priorité Haute

Isolation des validations Secrétariat

Problème :

Toutes les demandes sont visibles dans un tableau unique.

Solution envisagée :

Restreindre les accès selon les droits attribués lors de l'authentification.

Priorité Moyenne

Sécurisation des URLs

Problème :

Les URLs sont accessibles directement sans contrôle.

Exemples :

/conges/chef/

/conges/secretariat/

/conges/dg/

Conséquences :

Toute personne connaissant l'URL peut accéder aux interfaces de validation.

Solution envisagée :

Mettre en place :

- authentification ;
- permissions ;
- contrôle des rôles ;
- protections Django (login_required, groupes, permissions).

Priorité Haute

Calendrier des absences par responsable

Problème :

Le calendrier affiche actuellement l'ensemble des demandes.

Solution envisagée :

Après intégration des comptes utilisateurs :

- filtrer les événements selon le responsable ;
- afficher uniquement les collaborateurs rattachés au chef connecté.

Priorité Moyenne

Notifications intranet

État actuel :

Les emails automatiques fonctionnent correctement.

Reste à faire :

Mettre en place les notifications directement dans l'intranet afin de compléter les emails.

Priorité Faible

D.23 Résumé de la documentation modification quand l'authentification sera actif sur l'intranet

Objectifs

Ce document décrit l'ensemble des modifications à réaliser dans l'application **conges/** lorsque le système d'authentification de l'intranet Sectronic sera disponible.

L'objectif est de passer d'une application ouverte et sans contrôle d'accès à une application sécurisée, isolée par utilisateur et par rôle, sans changer la logique métier existante.

Rôles concernés

Rôle	Description
employe	Tout collaborateur qui soumet des demandes d'absence
chef	Chef de service, valide en premier
secretariat	Secrétariat, valide en second
dg	Direction Générale, validation finale

Fichiers impactés

Fichier	Impact
models.py	Ajout du lien vers l'utilisateur Django
views.py	Protection des vues et filtrage par utilisateur
forms.py	Suppression des champs saisis manuellement
urls.py	Protection des vues
settings.py	Paramètres d'authentification
templates	Adaptation de l'affichage selon les rôles

Étapes prévues

Étape 1 – Vérification de la hiérarchie

Objectif :

- Vérifier l'existence des groupes :
    - chef
    - secretariat
    - dg
- Vérifier que chaque utilisateur possède :
    - first_name
    - last_name
    - email

Étape 2 – Modification du modèle DemandeAbsence

Objectif :

Associer chaque demande à l'utilisateur Django connecté afin de permettre le filtrage des données.

Fichier concerné :

- conges/models.py

Étape 3 – Création du filtre has_group

Objectif :

Permettre la vérification des groupes directement dans les templates Django.

Fichiers concernés :

- conges/templatetags/
- conges_tags.py

Étape 4 – Modification du formulaire

Objectif :

Supprimer les champs :

- nom
- prénom
- email
- email_chef

Ces informations seront récupérées automatiquement depuis le compte utilisateur.

Étape 5 – Sécurisation des vues

Objectifs :

- Ajout de `@login_required`
- Ajout de contrôles d'accès par rôle
- Filtrage des demandes selon l'utilisateur connecté
- Protection des validations

Étape 6 – Modification des templates

Objectifs :

- Suppression des champs de saisie devenus inutiles
- Affichage automatique des informations utilisateur
- Adaptation de la navigation selon les rôles

Étape 7 – Paramétrage Django

Fichier :

- settings.py

Paramètres :

- LOGIN_URL
- LOGIN_REDIRECT_URL
- LOGOUT_REDIRECT_URL

Étape 8 – Calendrier par équipe

Objectif :

Limiter l'affichage des absences aux collaborateurs rattachés au responsable connecté.

Étape 9 – Tests de validation

Tests prévus :

- Contrôle des accès
- Vérification des rôles
- Isolation des données
- Validation du workflow
- Vérification des PDF
- Vérification des notifications

Points d'attention :

- Conservation des demandes historiques.
- Migration progressive des comptes utilisateurs.
- Gestion automatique des emails de validation.
- Filtrage des calendriers par équipe.
- Gestion des permissions Django.